University of Hertfordshire UH

PROJECT REPORT

BEng

Name: Sam Yarmohammad Touski Donadelli

Supervisor: Maryam Banitalebi Dehkordi

Lane Detection and Following for a Simulated Mobile Robot

27 March 2026

BACHELOR OF ENGINEERING DEGREE/DEGREE WITH HONOURS IN ROBOTICS AND ARTIFICIAL INTELLIGENCE

Department of Engineering
School of Physics, Engineering and Computer Science
University of Hertfordshire

LANE DETECTION AND FOLLOWING FOR A SIMULATED MOBILE ROBOT

Report by
SAM YARMOHAMMAD TOUSKI DONADELLI

Supervisor
MARYAM BANITALEBI DEHKORDI

Date
27 OF MARCH 2026

DECLARATION STATEMENT

I certify that the work submitted is my own and that any material derived or quoted from the published or unpublished work of other persons has been duly acknowledged (ref. UPR AS/C/6.1, Appendix I, Section 2 – Section on cheating and plagiarism).

I confirm that any study conducted as part of my project that involved human participants has been approved in accordance with UH Ethical regulations and the protocol number(s) is/are:

LIST ANY ETHICAL APPROVAL REFERENCE NUMBERS HERE OR TYPE “N/A” IF NOT APPLICABLE

Student Full Name: SAM YARMOHAMMAD TOUSKI DONADELLI

Student Registration Number:

Signed: …Sam Yarmohammad Touski Donadelli…

Date: ……27/03/2026……

ABSTRACT

The report shows the complete development process of a lane detection system which includes implementation and evaluation for a virtual robot that moves through a three-dimensional space created with Unity 3D. The project solves the basic problem of autonomous navigation through perception by building a control system which allows a mobile robot to find lane edges and determine its position between lanes while creating instant steering adjustments. The perception system includes ten downward looking raycasts which function like reflective line sensors to detect lane marks on two different physics layers. The system achieves strong lane edge detection through median filtering which operates on its sensor network. The system takes three samples from the lane ahead of the robot to help with both immediate adjustments and future path planning. The control system development process started with a PD controller which the supervisor recommended to expand into a five-term system that includes Proportional, Integral, and Derivative control for lateral error correction and heading correction and a feedforward term from the pure pursuit algorithm. The control system receives its integral term which includes three safety functions that block windup through clamping and perform leaky integration and block system entry through specific validation rules. The obstacle avoidance system operates by using nine sphere casts which form a fan pattern to identify obstacles before it performs a four-step process to go around them.

The evaluation of performance occurred through three different track layouts which included a straight track and an S-shaped track and a track that needed obstacle navigation. The straight-line track achieved a mean absolute error of 0.08% normalised, unitless error, the S-curve track produced a mean error of 3.6%, and the obstacle avoidance track maintained a mean error of 5% while successfully navigating around a stationary obstacle. The results show that each stage of the iterative development process brought better performance results which led to a final controller that maintains stable lane tracking through different track layouts.

ACKNOWLEDGEMENTS

My project supervisor Maryam deserves my complete appreciation because she provided excellent guidance and support throughout my entire project. The feedback she provided helped me enhance both my technical work and report quality through continuous improvement. The feedback I received from her helped me improve my technical work and report quality through continuous development. The feedback I received from her helped me improve my technical work and report quality through continuous development. Her support helped me maintain my focus through all the difficult phases which involved developing and testing my work. I received valuable suggestions from her which helped me create an extended controller architecture. She provided specific instructions about how to optimize the project scope which I found extremely helpful.

The Department of Engineering staff at University of Hertfordshire staff members provided resources and support which I received during my project work. My work would not have been possible without the understanding and support which my family members and friends gave me throughout this entire process.

TABLE OF CONTENTS

LIST OF FIGURES

GLOSSARY

AGV — Automated Guided Vehicle

AV — Autonomous Vehicle

CSV — Comma-Separated Values

IPM — Inverse Perspective Mapping

MAE — Mean Absolute Error

PD — Proportional-Derivative controller

PID — Proportional-Integral-Derivative controller

RMS — Root Mean Square

RMSE — Root Mean Square Error

Anti-Windup — A technique to prevent the integral term in a PID controller from accumulating excessively when the control output is saturated.

FixedUpdate — A Unity callback executed at fixed time intervals (default 0.02s) used for physics calculations.

Heading Error — The angular difference between the robot's forward direction and the direction toward the lane centre ahead.

Hysteresis — Using different thresholds for entering and exiting a state to prevent rapid switching on borderline conditions.

Lateral Error — The perpendicular distance between the robot's position and the lane centre, normalised by the lane half-width.

LayerMask — A Unity mechanism for filtering which physics layers a raycast can detect.

PhysX — NVIDIA PhysX physics engine integrated within Unity.

Pure Pursuit — A geometric path-tracking algorithm that calculates steering required to follow a circular arc to a lookahead point.

Raycast — A computational method projecting an invisible ray from a point in a direction to detect collisions.

Rigidbody — A Unity physics component giving a game object mass, drag, and the ability to respond to forces.

Unity — Unity 3D game engine (version 6000.3.5f2 used in this project).

1. Introduction

1.1 Project Background and Justification

Autonomous mobile robots need to master three essential capabilities which include self-localization and navigation planning and movement execution control (Siciliano and Khatib, 2016). The fundamental capability to detect and follow lanes serves as the base for all operations which include warehouse AGVs (Ullrich, 2015a) and modern car lane-keeping assist systems (Winner *et al.*, 2016). This project established a simulation environment to build a lane-following robot through feedback control basics while I tested the limits of basic perception-control systems.

Simulation was selected as the working method to avoid using actual hardware equipment. The Unity 3D platform through its built-in NVIDIA PhysX engine enables developers to create rigid body dynamics while running physics simulations at 50 Hz through FixedUpdate and developers can use C# scripting to test different control systems quickly (Juliani *et al.*, 2018). The simulation environment eliminated all expenses and physical space requirements and equipment damage potential which physical prototyping would have caused. The system produces exact duplicate results when users start with the same initial parameters because PhysX produces identical results from identical initial conditions (Farley, Wang and Marshall, 2022). The system operates with realistic components, yet PhysX fails to simulate actual tire deformation and surface bumps and robot sensor disturbances which the report identifies as its main weakness.

The project uses downward-facing raycasts to create a simulated reflective infrared sensor array which follows the same basic design as industrial line-following robots (Pakdaman and Sanaatiyan, 2009). The system uses raycast sensors which detect when they cross over lane marking signs. The design preserves basic elements without any additional components. The project aimed to develop solutions for computer vision problems because camera-based lane detection through (Canny, 1986) edges and perspective transforms and deep learning methods already exist, but they demand significant computing power (Janai *et al.*, 2020a) The project solves the control problem through sensor signals which remain free from noise and maintain complete determinism to show how the robot should move based on its knowledge of lane boundaries.

The project started with a requirement to develop a Proportional (P) controller. The initial tests revealed that P-only control caused extreme oscillations when running at normal speeds, so the supervisor suggested starting with PD control before adding integral action and heading correction and feedforward through pure pursuit and finally an obstacle avoidance state machine. Each new element was introduced after discovering specific problems in the previous version which became the core value of the iterative development work. The final controller system includes five steering components (P + I + D + heading + feedforward) but the PD and heading components perform most of the work. The integral

component stays under heavy control while it builds up only during specific situations to function as a straight-line bias adjustment system instead of following standard, I term behaviour. The system operates through four separate phases of obstacle avoidance which disable right-side lane sensors during vehicle movements while using side-facing sphere casts to maintain safe following distances.

1.2 Projects Aims and Objectives

This project establishes a perception-based control system which I designed, built, and tested to operate a virtual mobile robot that follows lane markings in Unity 3D simulation space. The project needs to show how testing different controller versions starting with a basic PD controller and advancing to a multi-term system with obstacle avoidance produces better system performance results.

The work requires five objectives which form its complete structure:

- **Objective 1 —** Environment Construction: Build a 3D simulation environment in Unity with realistic physics (NVIDIA PhysX) and at least three track layouts of increasing difficulty: a straight track for baseline testing, an S-curve track for evaluating cornering behaviour, and a track with a stationary obstacle for avoidance testing.

- **Objective 2 —** Perception System Design: Develop a sensor system using Unity's Raycast API to simulate downward-facing reflective sensors. The system must detect left and right lane markings on separate physics layers, estimate the robot's lateral position relative to the lane centre using median-filtered readings, and provide a continuous error signal to the controller at each 0.02-second physics timestep.

- **Objective 3 —** Control System Development: The development process requires multiple controller stages which begin with PD control and then add integral action and heading correction and finally feedforward anticipation through pure pursuit. Each addition must be justified by evidence which proves each new feature solves a particular problem that existed in the previous system version. The process of controller parameter adjustment requires testing multiple times to find stable tracking solutions which produce acceptable levels of oscillations.

- **Objective 4 —** Obstacle Avoidance Implementation: The system needs sphere cast obstacle detection which scans a fan-shaped area to perform a state machine based on Dodging → Passing → Merging sequence for executing safe lane changes around fixed obstacles while maintaining proper lane positioning.

- **Objective 5 —** Performance Evaluation: The final controller will undergo quantitative evaluation through automated telemetry data collection which operates at 50 Hz to record metrics that include mean absolute error (MAE), root mean square error (RMSE), maximum deviation,

average speed, and off-track events. The research results become available through MATLAB-generated figures which also include tables.

1.3 Risk Assessment and Mitigation

The project operates through complete simulation which removes all physical dangers that normally exist when working with robotic equipment in workshops or handling electrical parts or moving mechanical components. The three remaining risks consisted of computing problems and standard operational procedures which produced various challenges.

The most serious risk was data loss. Unity projects become damaged when the editor stops working during a save operation which results in lost memory-based telemetry data after application closure. This was mitigated through a strategy if backing up Unity project folders at scheduled intervals and the PerformanceLogger system which stores CSV files in short writing operations that continue to save data until Unity stops working during testing.

The second risk was physics instability. PhysX produces unpredictable results when robot setups enable impossible movements such as robot tip-overs because of unbalanced force application. The solution required the Rigidbody to stop rotation on X and Z axes while it maintained steering control through Y-axis rotation. Physics interpolation was enabled to achieve better rendering results and all control outputs to stay within physically possible boundaries. ForceMode.Acceleration was used to apply forces which kept the robot's acceleration independent from its weight while preventing unexpected system responses when they modified the robot's weight during system tuning.

The third risk is the validity gap between simulation and reality. The PhysX engine creates a suitable representation of rigid body physics, but it lacks the ability to simulate tyre slip and surface texture and sensor noise and actuator delay and environmental disturbances. The controller shows higher performance values than what a real robot would perform. The report explains this restriction through its entire document and Chapter 5 establishes sim-to-real transfer as the main research topic which needs future investigation.

This project did not require ethical approval because it did not involve any human subjects or animal experiments or personal information collection.

1.4 Report Structure

The report follows the six-chapter structure which the Department of Engineering project handbook requires.

The first chapter of this report establishes the background elements of the project while it establishes specific project targets and identifies potential risks. The second chapter analyses past research about self-driving systems and lane recognition methods and PID control operations and pure pursuit path following and obstacle detection and robotic simulation studies. The third chapter explains the complete methodology through its description of the simulation environment and perception system architecture and the complete five-term controller development process which started from PD control and the obstacle avoidance state machine and data collection methods. The fourth chapter delivers numerical test results which emerged from running tests on three different track layouts. The chapter includes detailed analysis of error metrics and speed profiles and sensor reliability and controller term contributions. The fifth chapter establishes conclusions from the research study while it assesses each goal based on study results and presents an honest evaluation of research limitations and specific recommendations for future research directions. The sixth chapter includes a project management assessment which shows the initial Gantt chart against actual work completion and evaluates how expanded project requirements impacted the project and includes a summary of the knowledge gained through this experience.

2. Literature Review

The chapter presents a detailed analysis of theoretical elements and previous studies which establish the foundation for all design choices made in this project. This chapter examines five essential areas which include autonomous mobile robot navigation and lane detection methods and feedback control theory and path tracking algorithms and robotics research based on simulation studies. The document presents essential principles in each section which demonstrate how this project connects to existing knowledge in the field.

2.1 Autonomous Mobile Robot Navigation

Research into autonomous mobile robot navigation in structured environments dates to the mid-1980s, when (Moravec and Elfes, 1985) introduced occupancy grid mapping using sonar data. Researchers have studied this field since then to develop systems which operate in warehouse logistics and hospital delivery and agricultural inspection and planetary exploration (Siegwart, Nourbakhsh and Scaramuzza, 2011).

(Siciliano and Khatib, 2016) describe the standard architecture as three layers: perception (sensing the environment), planning (deciding a path), and control (executing it). The planning layer in a lane-following system functions as a replacement using lane markings which form the actual track. The system forms a perception-control loop because it needs to detect lane edges and measures robot position from centre before applying steering corrections (Dudek and Jenkin, 2010).

The project needs to work with a robot system which moves forward but cannot slide sideways because it must steer to change its direction. The kinematic relationship between forward velocity v, angular velocity omega, and turning radius R is:

Equation 2-1

$$\omega = \frac{v}{R}$$

The design requires robots to perform longer turns at elevated velocity levels because their speed control system needs to maintain angular velocity. The controller system requires speed-dependent gain scaling along with automatic speed reduction in curves because robots need to execute wider turns at elevated speeds to keep their angular velocity constant (Klancar *et al.*, 2017).

2.2 Lane Detection Techniques

2.2.1 Reflective Sensor Arrays

The robot uses infrared reflective sensors which it mounts under its chassis to perform lane following according to (Pakdaman and Sanaatiyan, 2009). These emit infrared light and measure the reflectance difference between a dark line and a light surface. The robot determines its sideways distance from the line through its horizontal sensor array which contains multiple sensors.

The method needs minimal computer power to work because it generates exact results from either binary or analogue data without needing to process any images. The industrial space continues to support AGVs which operate through painted floor markings according to (Ullrich, 2015b). The system faces its main obstacle because it can only sense objects directly below the robot which means operators cannot see what lies around the next bend.

2.2.2 Camera-Based Lane Detection

Camera-based systems extract richer information. The standard process transforms images into HSL or HSV colour spaces which highlight lane markers before applying Canny edge detection (Canny, 1986) and performing perspective transformation for bird's-eye view acquisition (Hillel *et al.*, 2014). The Hough Transform then performs line fitting on these transformed images. The instance segmentation method of (Neven *et al.*, 2018) applies deep learning to separate lane markings during difficult situations which include shadowed areas and damaged roads and blocked views.

The methods deliver strong results, but they create multiple challenges which include camera calibration requirements and illumination sensitivity and high computational demands and large requirements for annotated training data (Janai *et al.*, 2020b) The project purpose meant the focus should be on control system behaviour rather than perception system complexity.

2.2.3 Raycast-Based Sensing Simulation

The perception system developed in this project uses Unity's Physics.Raycast to simulate downward-facing sensors (Unity Technologies, 2024c). Each raycast acts as a virtual reflective sensor: it fires a ray downward and reports whether it hit a lane marking, and if so, at what offset. By assigning left and right lane markings to separate physics layers (LeftLane and RightLane) and using LayerMask filtering, the system can distinguish between left and right boundaries without any colour processing.

This is a deliberate simplification. Real reflective sensors introduce noise, and real lane markings are imperfect. Unity raycasts produce clean, deterministic outputs (Juliani *et al.*, 2018), which means the controller's performance numbers are optimistic compared to a physical implementation. The

advantage is that the error signal fed to the controller is reliable enough to study control behaviour in isolation, without needing to debug perception failures at the same time.

2.3 Feedback Control Theory

The steering system receives its mathematical framework from feedback control which operates as its fundamental system. The robot tracks its side-to-side position as the process variable while it uses the lane centre as its target and the steering command functions as the system's control response. The three basic control actions which form the PID controller structure receive their explanation in the subsequent sections. This control system remains the most common industrial automation control system according to (Åström and Murray, 2021).

2.3.1 Proportional Control

The proportional controller generates a corrective output which directly links to the present error magnitude.

Equation 2-2

$$u(t) = Kp \times e(t)$$

The robot moves right under controller guidance because it detects positive errors when the robot veers toward left. The system requires larger correction amounts to fix bigger drifts which occur in the system (Ogata, 2010). The two main drawbacks of proportional control have been proven through research studies. The system reacts to changes after they occur because it lacks the ability to predict future modifications. The system generates an ongoing steady state offset when it encounters continuous disturbances because it needs a non-zero error value to produce any corrective response (Franklin, Powell and Emami-Naeini, 2015). The initial testing of this project showed P-only control caused continuous oscillations when operating above 1 m/s which led to the implementation of derivative control.

2.3.2 Derivative Control

The derivative term opposes rapid changes in error:

Equation 2-3

$$u_{d(t)} = Kd \times \left(\frac{de}{dt}\right)$$

The derivative term produces a braking effect which stops the robot from exceeding its target position while it moves toward the lane centre and error values decrease. The robot uses its movement away

from centre position to strengthen its proportional control which enables it to respond more quickly (Nise, 2020). The combined PD control law is:

Equation 2-4

$$u(t) = Kp \times e(t) + Kd \times \left(\frac{de}{dt}\right)$$

The main practical issue with derivative control is noise amplification: differentiation magnifies high-frequency components in the error signal. The standard approach requires users to apply a low-pass filter or exponential smoothing to the derivative calculation based on the work of (Åström and Hägglund, 2006).

Equation 2-5

$$d_{smooth(n)} = \alpha \times d_{smooth(n-1)} + (1-\alpha) \times d_{raw(n)}$$

Higher values of the smoothing factor alpha (between 0 and 1) produce smoother signals but introduce more lag. This trade-off was tuned empirically during the development.

2.3.3 Integral Control and Full PID Controller

The system needs the accumulation of error values from the integral term to reach a point where it removes the constant offset from the system.

Equation 2-6

$$u_{i(t)} = Ki \times \int_0^t e(\tau)d\tau$$

The system computes the sum which gets updated during every new time interval after it converts the continuous data into discrete values:

Equation 2-7

$$integralAccumulator \mathrel{+}= e(n) \times \Delta t$$

The complete PID control system merges its three separate components into a single control mechanism:

Equation 2-8

$$u(t) = Kp \times e(t) + Ki \times \int_0^t e(\tau)\, d\tau + Kd \times (de/dt)$$

The integral component of the system contains the standard problem which people call integral windup because the robot produces high integral values when it moves through curved paths which produce

continuous tracking deviations that stem from path design rather than control system faults. The accumulated value leads to an overcorrection which drives the vehicle away from its lane centre position. (Visioli, 2006) presents three methods to fight windup which include setting fixed limits for accumulator values and stopping the integration process during short-term changes and using back-calculation to decrease accumulator values when control signals reach their maximum limit. The Ziegler-Nichols tuning method (Ziegler and Nichols, 1942) shows how process control systems need PID gain settings to prevent oscillations which became essential for controller tuning during this project. The project contains a strictly controlled integral term which activates only when the system identifies both lane boundaries while the robot stays outside curves and avoids detection mode and recovery mode. The system functions as a bias-correction tool for straight-line driving instead of working as a standard integral controller. Chapter 3 presents the design rationale which underpins the conservative approach of this study.

2.4 Pure Pursuit Path Tracking

While PID control reacts to errors after they occur, pure pursuit is a feedforward algorithm that anticipates the steering needed to reach a point ahead on the path. Developed at Carnegie Mellon University for autonomous vehicle navigation, it calculates the curvature of a circular arc from the robot's current position to a lookahead point on the intended path (Coulter, 1992):

Equation 2-9

$$\boldsymbol{\kappa = 2x / L^2}$$

where kappa is the curvature (inverse of turning radius), x is the lateral offset of the lookahead point from the robot's heading, and L is the distance to the lookahead point. The required angular velocity is then:

Equation 2-10

$$\omega = v \times \kappa$$

The key tuning parameter is the lookahead distance L. A short lookahead tracks the path closely but tends to be twitchy; a long lookahead produces smoother steering but cuts corners on sharp curves. (Snider, 2009) recommends scaling the lookahead distance with speed, looking further ahead at higher speeds for stability, and closer at lower speeds for accuracy. (Thrun, Burgard and Fox, 2006a) note that pure pursuit's geometric simplicity makes it efficient enough for real-time execution on resource-constrained platforms.

In this project, pure pursuit is not used as the primary steering algorithm. Instead, it provides a feedforward term that supplements the PID feedback controller. The idea is that PID handles error correction while pure pursuit handles curve anticipation, combining the precision of feedback control with the smoothness of geometric path tracking.

2.5 Obstacle Detection and Avoidance

Obstacle avoidance is a fundamental capability for any mobile robot operating in an environment where objects may block their intended path.

(Khatib, 1986) introduced the potential field method, which treats obstacles as repulsive force sources and the goal as attractive. The robot follows the combined gradient. While elegant, this method suffers from local minima, the robot can get trapped in stable positions between opposing (Siegwart, Nourbakhsh and Scaramuzza, 2011). (Borenstein and Koren, 1991) addressed this with the Vector Field Histogram (VFH), which builds a polar histogram of obstacle density and selects gaps wide enough for safe passage. (Lumelsky and Stepanov, 1987) proposed the Bug algorithms, simple reactive strategies where the robot follows obstacle edges until a route opens toward the goal.

For a lane-following system, obstacle avoidance is more constrained than in open navigation: the robot must stay within lane boundaries while manoeuvring around the obstacle. (Gonzalez *et al.*, 2016) describe this as a local motion planning problem that decomposes into phases: detection, evasion, passing, and re-entry. (Rajamani, 2012) discusses the vehicle dynamics of lane changes, noting that controlled lateral movement requires careful speed management to maintain stability.

The phased approach influenced the design of this project's avoidance system, which uses a four-state finite state machine (None, Dodging, Passing, Merging). The current implementation only dodges to the left and only handles stationary obstacles, limitations that are acknowledged and discussed in Chapter 5.

2.6 Simulation-Based Robotics Research

Simulation has become an essential tool in robotics research, as physics engines have reached sufficient fidelity to produce useful results. (Farley, Wang and Marshall, 2022)compared four major robotics simulation platforms and confirmed that modern physics engines can replicate wheeled robot motion with acceptable accuracy, though they recommend validating simulation results with physical hardware wherever possible.

Unity 3D is widely used as a simulation platform in robotics and AI research. (Juliani *et al.*, 2018) describe Unity's ML-Agent's toolkit, which uses Unity's physics and rendering capabilities to train reinforcement learning agents. While this project does not use machine learning, it relies on the same PhysX engine for rigid body dynamics, collision detection, and deterministic fixed-timestep simulation (Unity Technologies, 2024a).

(Ivaldi *et al.*, 2015) surveyed robotics researchers on simulator selection and found that ease of use, documentation quality, and community support were rated as the most important factors, often above raw physics fidelity. Unity scores well on all three criteria, which is why it was chosen over more specialised robotics simulators like Gazebo or Webots that offer features not required for this project.

2.7 Summary

The literature review established the theoretical and practical foundations for this project. Reflective sensor arrays provide a simple, computationally efficient perception approach well suited to controlled environments. PID control theory offers a complete and well-understood framework for steering correction, with established techniques for handling practical challenges like derivative noise and integral windup. Pure pursuit provides a complementary feedforward mechanism for path tracking that works particularly well alongside reactive PID control. Phased state machine architectures offer a structured approach to obstacle avoidance within lane-following constraints. Unity 3D provides a simulation platform that balances physics fidelity with development accessibility, supported by a substantial body of research validating its use for robotics applications.

The following chapter describes how these theoretical elements were combined into a working system, documenting every design decision and its justification.

3. Methodology, Design and Implementation

This chapter walks through how the lane-following system was designed and built, covering the decisions made at each stage and the reasoning behind them. The structure follows the same signal flow the robot executes every physics timestep: the environment provides the track, the perception system reads the lane, and the controller generates steering.

3.1 Simulation Environment Design

3.1.1 Platform Selection Configuration

The simulation was built in Unity 3D version 6000.3.5f2. Unity was chosen for three practical reasons that the literature review highlighted: its integrated NVIDIA PhysX engine handles rigid body dynamics well enough for mobile robot simulation (Farley, Wang and Marshall, 2022), its FixedUpdate loop runs at deterministic intervals which is essential for getting reproducible results from control experiments (Unity Technologies, 2024a), and its C# scripting environment made it easy to try things quickly and iterate on designs (Juliani *et al.*, 2018).

The physics timestep was set to the default 0.02 seconds, giving 50 updates per second. At the robot's top speed of 4 m/s this means it moves about 0.08 metres between updates, which is comfortably within what the sensor array can resolve. The robot's Rigidbody was set up with rotation frozen on the X and Z axes (RigidbodyConstraints.FreezeRotationX and FreezeRotationZ), leaving only Y-axis rotation free for steering (Unity Technologies, 2024b). In practical terms this stops the robot from tipping or rolling while keeping the non-holonomic steering behaviour that (Siegwart, Nourbakhsh and Scaramuzza, 2011) describe for car-like platforms. Physics interpolation was turned on so that the rendering looks smooth between discrete physics steps. The visual car model comes from the Racing Cars Pack 1 asset (Unity Asset Store, 2024b) and sits on top of the physics body with a configurable offset to line up the visual mesh with the actual collision geometry.

Image 3.1: Robot model showing Rigidbody collision geometry and sensor debug visualisation in Unity editor.

3.1.2 Environment Construction

The simulation environment was put together using a few Unity Asset Store packages. The SimplePoly City package (Unity Asset Store, 2024a) provides low-polygon buildings, trees, street furniture, and terrain. The low-poly style was a practical choice: it keeps rendering lightweight, so the physics simulation holds a steady frame rate without the GPU becoming a bottleneck.

The road network uses the Road System package (Unity Asset Store, 2024c), which comes with prefabricated road segments including straights, curves of various radii, and intersections. Because everything is modular it was straightforward to snap different segments together to create the three test tracks. The default road markings were swapped out for custom materials on separate physics layers, described in Section 3.1.4.

Image 3.2: Word Building

3.1.3 Track Design

Three track layouts were built, each harder than the last. Every track uses road segments with a standard width of 5 metres and a 4-metre gap between the left and right lane markings, which matches the controller's expectedLaneWidth setting.

The straight-line track is the simplest: zero curvature, no obstacles. Any lateral error on this track comes from the initial acceleration phase as the robot finds the lane centre from wherever it starts. This track is just a sanity check for steady-state accuracy and convergence.

The S-curve track is the hardest test for the controller. It alternates left and right curves with varying radii, forcing rapid steering reversals at each transition. (Corke, 2017) identifies exactly this kind of alternating curvature as a key challenge for wheeled robot controllers. It also puts the anti-windup mechanisms under pressure, since integral error from one curve could easily carry over and cause problems in the next.

The obstacle avoidance track is a straight road with a stationary obstacle placed in the lane. The robot must spot it, shift left to get around it and then merge back to the lane centre. This tests whether the avoidance state machine and the lane-following controller can work together without fighting each other.

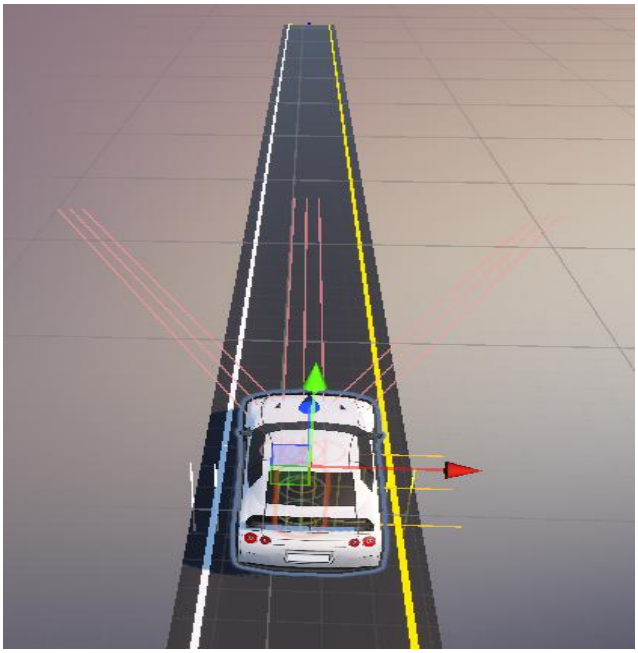

Image 3.3: Straight-Line Track

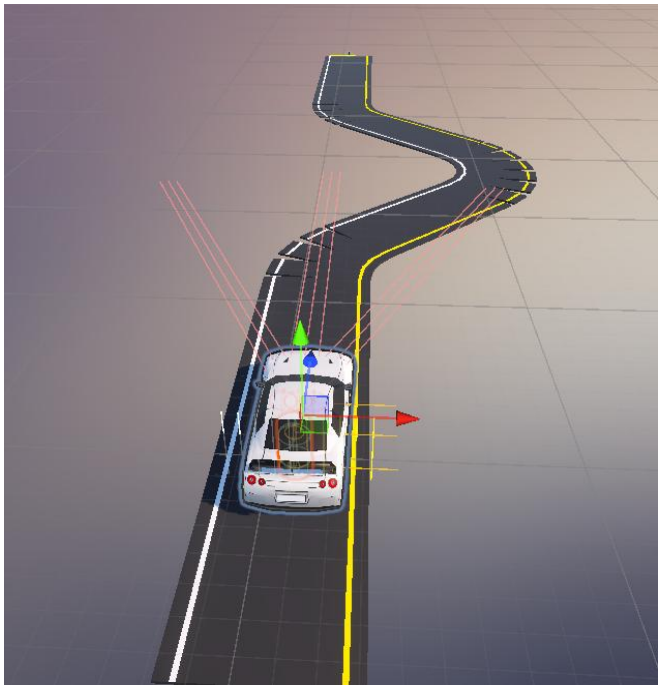

Image 3.4: S-curve Track

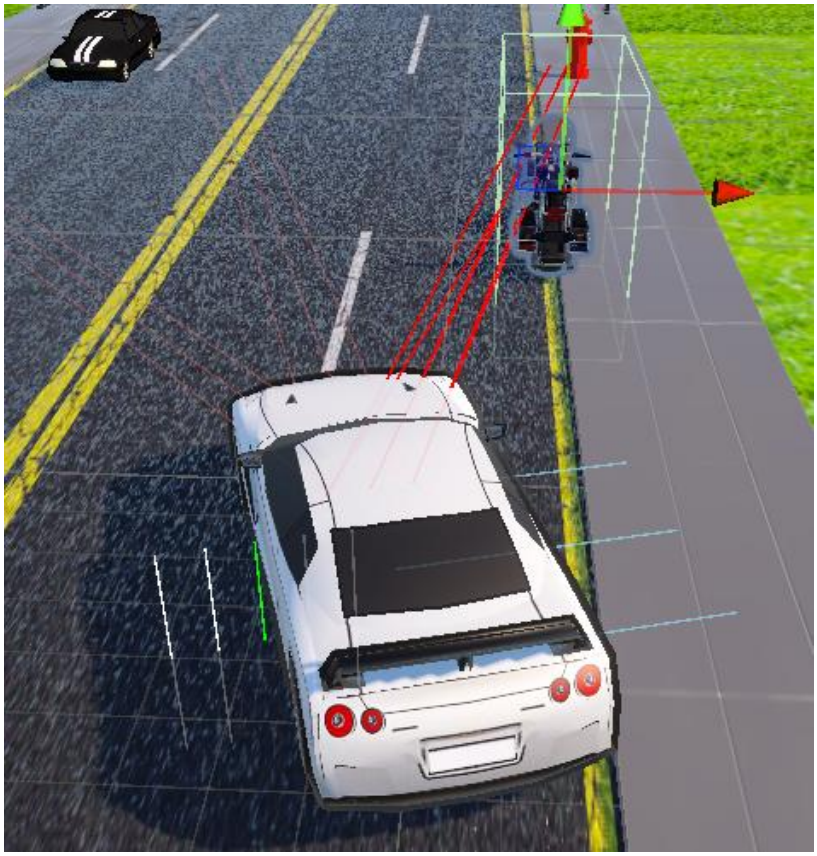

Image 3.5: Obstacle Avoidance Track

3.1.4 Lane Marking Design

The lane boundaries use two separate materials on distinct Unity physics layers: white for the left edge (LeftLane layer) and yellow for the right edge (RightLane layer). This lets the perception system tell left from right using LayerMask filtering when raycasting (Unity Technologies, 2024c), which is essentially the same principle as physical line-following robots that use colour-selective sensors (Pakdaman and Sanaatiyan, 2009). It replaces colour-based image processing with a much simpler layer-based check while keeping the core idea of lane-specific detection intact.

3.2 Perception System Design

3.2.1 Sensor Architecture

The perception system fires downward facing raycasts to find lane markings, simulating the reflective infrared sensor arrays described in Section 2.2.1. Rays point straight down from positions spread across the robot's width at a configurable forward distance.

There are five sensors per side (sensorsPerSide = 5), spread from an inner offset of 0.3 metres to an outer offset of 2.5 metres (sensorSpread = 2.5). Each one calls Physics.Raycast pointing downward

with a 3.0-metre detection range, filtered by LayerMask so it only picks up the correct lane layer. The reason for having five rather than one or two is redundancy: if one sensor misses the marking because of a gap in the road surface, the others still see it.

Instead of averaging all the hit positions or just taking the closest one, the system picks the median hit offset from whichever sensors detected the edge on that side. Median filtering handles outliers much better than averaging (Gonzalez and Woods, 2018). For instance, if the left sensors return offsets of [0.5, 0.8, 1.1, 1.1, 2.3] metres, the median is 1.1, which correctly ignores both the too-close and too-far readings.

3.2.2 Lane Centre Estimation

How the system works out the lane centre depends on how many edges it can see, because sensors lose detection often during curves and transitions.

When both edges are visible, the centre is simply the midpoint between the two median hit positions. The lane half-width is calculated by blending the expected value with the actual measurement:

Equation 3-1

$$halfWidth = lerp(expectedHalf, measuredHalf, \beta)$$

where beta is widthBlendToMeasured, set to 0.7 in the final configuration. Setting this above 0.5 means the system trusts actual measurements more than the expected value. This helps when road geometry varies slightly, while still falling back toward the expected width if the measured value looks odd during a detection error.

When only one edge is visible, the system guesses where the centre is by projecting the expected half lane width from whichever edge it can see. So, if only the left edge shows up, the centre is estimated as that position plus 2.0 metres to the right. This is less accurate than using both edges, but it keeps the robot roughly on track when one boundary disappears during sharp turns or track joints.

When neither edge is visible, the system enters recovery mode. The robot holds its last steering direction for up to five seconds (recoveryTimeout = 5), giving it a window to reacquire the markings. If detection comes back within that time, normal operation resumes. If it does not and keepMovingDuringRecovery is enabled (which it is), the robot keeps moving at its current heading rather than stopping dead.

3.2.3 Multi-Distance Sampling

One of the more important design decisions in the perception system is that it samples lane data at three different distances ahead of the robot, using the same sensor array layout each time but projected to different forward positions:

The near sample (sensorForwardDistance = 1.0 metres ahead) gives the primary lateral error for steering. Because it is close, the error reflects where the robot is right now rather than where it might be in the future.

The far sample (lookahead distance, ranging from 1.8 to 3.8 metres depending on speed) provides the aim point used for heading correction and the pure pursuit feedforward term. At higher speeds the robot covers more ground per timestep, so it needs to look further ahead to catch upcoming changes in time (Snider, 2009). The lookahead distance scales with speed:

Equation 3-2

$$lookahead = lerp(lookaheadMin, lookaheadMax, speed01 \times speedFactor)$$

where speed01 is the normalised speed (0 to 1) and speedFactor (1.2) controls how quickly the lookahead extends. During curves the lookahead is shortened by 30% so the robot tracks the curve shape more tightly instead of looking past it.

The prediction sample (predictionDistance = 6.0 metres ahead) gives early warning of curvature changes so the robot can start slowing down before it reaches the curve. This longest-range sample is only used for speed decisions. Using it for steering would be a bad idea because lane data from that far ahead is too noisy to steer from reliably.

3.3 Control System Design

The controller was built up incrementally. It started as PD control, and each additional term was added because the previous version had a specific problem that needed solving. This section describes the final architecture and explains what each part does and why it is there.

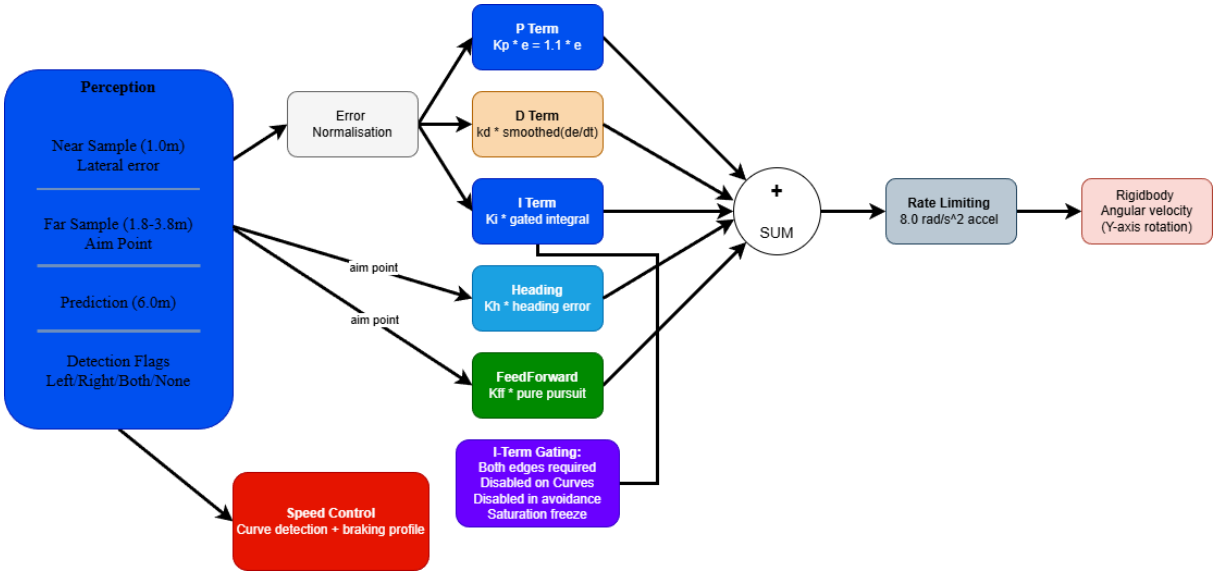

Image 3.6: Control system signal flow showing perception inputs, five steering terms, rate limiting, and speed control.

3.3.1 Error Normalisation and Smoothing

The raw lateral offset from the lane centre gets normalised by the lane half-width to produce a dimensionless error:

Equation 3-3

$$e = (centerLocal.x + avoidOffset) / halfWidthUsed$$

centerLocal.x is the lateral distance from lane centre in the robot's local frame. avoidOffset is a deliberate lateral shift used during obstacle avoidance (it stays at zero during normal driving). halfWidthUsed comes from Equation 3.1. The error is clamped between -1 and +1 so that bad lane detections cannot produce absurdly large values. Normalising the error this way is standard practice for mobile robot controllers because it produces a dimensionless ratio that works regardless of the actual lane width (Klancar *et al.*, 2017).

The raw error then goes through an exponential smoothing filter:

Equation 3-4

$$smoothedError = lerp(smoothedError, currentError, \alpha)$$

where $alpha = 1 - exp(-delta\_t / tau)$ comes from the physics timestep and the smoothing time constant tau (errorSmoothingTime = 0.2 seconds). Using an exponential formula rather than a simple linear blend means the smoothing behaves identically regardless of the timestep, which matters if the timestep ever changes (Åström and Hägglund, 2006).

3.3.2 Proportional Term

The proportional term is the simplest part of the controller. It steers in proportion to how far off-centre the robot is:

Equation 3-5

$$pTerm = Kp \times e$$

with $Kp = 1.1$. If the robot has drifted left (positive error), it steers right, and vice versa. Bigger drift means harder correction (Ogata, 2010).

A dead zone of 0.04 is applied on straight sections: errors smaller than 4% of the lane half-width get treated as zero. The point of this is to stop the controller from making tiny pointless corrections when the robot is already well centred and the "error" is just sensor noise. Dead zones are a standard PID refinement (Franklin, Powell and Emami-Naeini, 2015).

3.3.3 Derivative Term

The derivative term damps oscillations by pushing back against rapid error changes:

Equation 3-6

$$dTerm = Kd \times (1 + 0.3 \times speed/maxVelocity) \times smoothedDerivative$$

with $Kd = 0.4$. The derivative is worked out from consecutive error samples and then smoothed with an exponential filter (derivativeSmoothing = 0.85) to tame the noise that raw differentiation always amplifies (Section 2.3.2).

The factor $(1 + 0.3 * speed/maxVelocity)$ scales the derivative gain up at higher speeds. The reason is straightforward: the robot covers more ground per timestep when it is moving fast, so any oscillation is more dangerous and needs stronger damping. At maximum speed (4 m/s) the effective gain is 1.3 times Kd, which works out to 0.52. At rest it equals the base value of 0.4. During obstacle avoidance the derivative gets an additional boost from avoidKdBoost (2.5) because the rapid lateral shifts during dodging would otherwise produce quite a lot of wobbles.

3.3.4 Integral Term with Safety Mechanisms

The integral term is supposed to eliminate persistent steady-state offset by accumulating error over time:

Equation 3-7

$$integralAccumulator \mathrel{+}= e \times \Delta t$$

Equation 3-8

$$iTerm = Ki \times smoothed(integralAccumulator)$$

with $Ki = 0.08$. In textbook PID, this would gradually remove whatever small offset the P and D terms leave behind (Visioli, 2006).

In practice, this turned out to be the trickiest part of the whole controller. Letting the integral accumulate freely caused more problems than it fixed. The worst case was during curves: the robot spends several seconds with sustained lateral error (because the curve geometry makes some error unavoidable), and the integral dutifully accumulates all of it. Then when the curve ends and the road straightens out, the built-up integral produces a massive correction that overshoots the lane centre and triggers oscillation. This is the classic integral windup problem described in Section 2.3.3.

The solution was to be very conservative about when the integral is allowed to do anything at all:

Anti-windup clamping keeps the accumulator within [-0.45, +0.45], so there is a hard ceiling on how much it can build up no matter what.

Leaky integration bleeds the accumulator back toward zero at 0.35 per second. This means old, accumulated error fades on its own even without an explicit reset.

Conditional gating shuts off integration entirely during curves, obstacle avoidance, and recovery mode. The errors in those situations come from track geometry or deliberate manoeuvres, not from a persistent bias in the controller, so integrating them would do more harm than good.

Both-edge gating (integrateOnlyWhenBothEdges = true) only lets the integral accumulate when both lane edges are visible at once. If only one edge is detected the error estimate is less reliable, so the integral should not be trusting it.

Saturation-aware freezing (freezeIntegralWhenSaturated = true) stops the integral from growing when the steering output is already maxed out. Without this the integral keeps building up during saturation, and when the situation resolves the pent-up integral causes a big overshoot (Åström and Hägglund, 2006).

The integral output is also smoothed (integralSmoothingTime = 0.5 seconds) to prevent sudden jumps in steering when the accumulator value shifts.

The result is that the integral contributes very little to the actual steering in most situations. It mostly acts as a quiet bias-correction for straight-line driving where a small persistent offset might exist. The real steering work is done by P, D, heading correction, and feedforward. This is why calling the controller a “full PID” would be slightly misleading: the I term is present, but it is so heavily restricted that the system behaves more like a PD controller with heading and feedforward, plus a cautious bias corrector on top.

3.3.5 Heading Correction Term

The heading correction steers the robot toward where the road is going, rather than only reacting to how far off-centre it currently is:

Equation 3-9

$$headingError = atan2(aimPoint.x, aimPoint.z)$$

Equation 3-10

$$hTerm = Kh \times headingError$$

aimPoint is the lane centre position at the far sample distance, expressed in the robot's local coordinates, and $Kh = 1.5$. The atan2 function works out the angle between where the robot is pointing and where the aim point sits: if the road curves left ahead, the robot starts turning left before it has drifted off-centre.

The best analogy is how a person drives. You do not stare at the bonnet of your car and wait until you have already drifted before correcting. You look ahead at where the road goes and steer toward it. (Thrun, Burgard and Fox, 2006a) call this the difference between reactive control (fixing errors after they happen) and anticipatory control (preventing them). Without heading correction, the PD controller had to wait for lateral error to develop before it could respond to curves, which meant the first few metres of every curve produced a noticeable error spike. Adding heading correction cut those spikes significantly because the robot starts aligning with the curve before the error builds up.

The heading error is smoothed (headingSmoothingTime = 0.18 seconds) and clamped to plus or minus 0.7 radians (roughly 40 degrees), so it does not produce wild corrections on very tight bends.

3.3.6 Feedforward Term via Pure Pursuit

The feedforward term uses the pure pursuit algorithm from Section 2.4 to calculate anticipatory steering:

Equation 3-11

$$\kappa = 2x / (x^2 + z^2)$$

Equation 3-12

$$\omega\_ff = v \times \kappa$$

Equation 3-13

$$ffTerm = Kff \times smoothed(\omega\_ff)$$

x and z are the lateral and longitudinal components of the smoothed aim point in local coordinates, v is the forward speed, and $Kff = 0.55$. The output is smoothed (feedforwardSmoothingTime = 0.18 seconds) to avoid abrupt steering jumps at curve entries.

The feedforward term works differently from P, I, and D because it does not react to an error. Instead, it uses the geometric relationship between where the robot is and where the road goes to calculate what the steering should be. The robot starts turning before any error develops, which lowers the peak lateral deviation at curve entries (Coulter, 1992). Combined with the heading correction, it means the controller can handle curves proactively rather than always playing catch-up.

3.3.7 Combined Steering Output and Rate Limiting

All five terms get added together to produce the total steering command:

Equation 3-14

$$rawTurnRate = pTerm + iTerm + dTerm + hTerm + ffTerm$$

This combined output is clamped to the maximum turn rate (2.2 rad/s during normal driving, 1.2 rad/s during avoidance) and then run through two stages of rate limiting. The first stage caps how fast the target steering can change (maxTurnRateChangePerSecond = 8.0 rad/s squared), which prevents the controller from requesting instant steering reversals. The second stage caps how fast the actual angular velocity can change (maxAngularAcceleration = 7.0 rad/s squared), providing a final smoothing layer before the force is applied.

This two-stage setup is like how industrial servo systems separate command filtering from actuator dynamics (Åström and Murray, 2021). The practical effect is that the robot cannot make physically impossible steering moves, which produces more realistic and stable behaviour (Rajamani, 2012).

3.4 Curve Detection and Speed Control

The speed control system automatically slows the robot down when curves are coming. Curve detection uses hysteresis, meaning it takes a higher error threshold to enter curve mode (curveEnterThreshold = 0.1) than to leave it (curveExitThreshold = 0.06). This stops the system from flickering between modes on borderline sections (Åström and Murray, 2021). The robot also must stay below the exit threshold for a sustained period (curveExitHoldTime = 0.5 seconds) before curve mode turns off.

Predicted curvature is estimated by comparing lane centre positions at the near and far sample distances:

Equation 3-15

$$curvature \approx lateralDeviation \,/\, longitudinalDistance^2$$

If the distant lane centre has shifted a long way sideways relative to the nearby one, that means a sharp curve is coming.

Speed reduction uses three factors: predicted curvature, heading error magnitude, and lateral error magnitude. The system picks whichever factor is demanding the most braking, squares it to create a progressive profile (gentle curves barely slow down, sharp curves brake hard), and interpolates between the maximum and curve velocities:

Equation 3-16

$$demand = max(curvatureFactor, headingFactor \times 0.8, errorFactor \times 0.5)$$

Equation 3-17

$$targetSpeed = lerp(maxVelocity, maxCurveVelocity, demand^2)$$

Forward force is applied using Rigidbody.AddForce with ForceMode.Acceleration, which makes the acceleration independent of the robot's mass (Unity Technologies, 2024b). As the robot approaches its target speed the applied force tapers off, and if forward speed exceeds the target the system applies active braking. A minimum speed floor of 0.5 m/s prevents the robot from stalling completely during tight curves or recovery.

3.5 Obstacle Avoidance System

3.5.1 Obstacle Detection

Obstacles are detected using a fan of nine sphere casts arranged in three columns of three rays, spread 25 degrees to each side of centre (obstacleFanHalfAngle = 25). Physics.SphereCast uses a sphere radius of 0.45 metres, which sweeps a wider volume than a thin raycast and catches objects that a single line ray might miss (Unity Technologies, 2024d).

One detail that took some trial and error to get right was the cosine projection on hit distances. Angled rays travel a longer path than a straight-ahead ray to reach an obstacle at the same actual forward distance. Without correction, the wide-angle rays would report falsely large distances. The fix is simple:

Equation 3-18

$$forwardDist = hitDistance \times cos(rayAngle)$$

This makes all rays report consistent forward distances regardless of angle, which prevents the avoidance system from triggering too early on wide-angle detections.

The distance at which avoidance begins is calculated dynamically from the robot's current speed, considering reaction time, braking distance, and how much space the lateral shift needs.

3.5.2 Avoidance State Machine

The avoidance behaviour runs on a four-state finite state machine, following the phased approach that (Gonzalez *et al.*, 2016) describe:

In the None state the robot drives normally and the right lane sensors work as usual. When the forward sphere casts spot an obstacle within the dynamic start distance, the system switches to Dodging and immediately suppresses the right lane sensors. The right sensors get turned off because the robot is about to intentionally cross toward the right lane edge, and if the sensors stayed active, they would detect that edge and try to steer the robot back, fighting the avoidance manoeuvre.

In the Dodging state the robot's target path shifts leftward. The offset grows as the obstacle gets closer, calculated using inverse linear interpolation between the start and stop distances. The shift is clamped

so the robot stays inside the lane (laneEdgeSafetyMargin = 0.3 metres). Speed drops in proportion to how close the obstacle is. If the left path is also blocked and the robot is very close (within obstacleStopDistance = 2.5 metres), it performs an emergency stop. Once the side sensors pick up the obstacle alongside the robot, the system moves to Passing. If forward clears without the side sensors ever triggering, it skips ahead to Merging after a short confirmation wait.

In the Passing state the full lateral offset is held and speed stays reduced while the robot drives alongside the obstacle. Once the side sensors have been clear for 0.35 seconds (sideClearConfirmTime), the system transitions to Merging.

In the Merging state the lateral offset is smoothly brought back to zero using SmoothDamp over 0.85 seconds (mergeSmoothTime). When the robot is within 0.15 metres of the lane centre, normal driving resumes and the right lane sensors switch back on.

During the whole avoidance sequence the controller scales its gains differently. Kp drops to 30% of its normal value (avoidKpScale = 0.3) and Kh drops to 40% (avoidKhScale = 0.4). This stops the lane-following logic from fighting the deliberate lateral shift. At the same time Kd gets boosted to 2.5 times its normal value (avoidKdBoost = 2.5) to kill the oscillation that rapid lateral movements would otherwise cause. The integral term is disabled entirely during avoidance because there is no point accumulating error from a manoeuvre that is intentional.

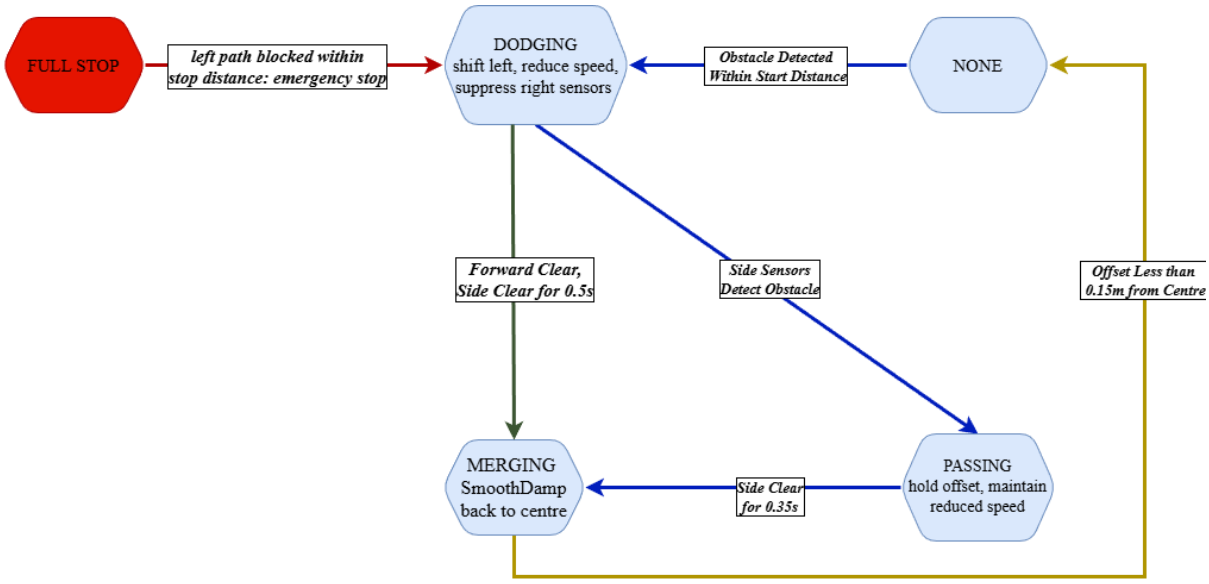

Image 3.7: Obstacle avoidance finite state machine showing states and transition conditions

3.6 Data Collection Methodology

The PerformanceLogger script records telemetry during every test run. It sits on the same GameObject as the RobotController and uses C# runtime reflection (System.Reflection) to read the controller's internal variables without touching the controller's code. This matters because a logging tool that modifies the thing it measures would produce misleading results (Sammut and Webb, 2017). With

reflection the logger is completely passive: it reads values but never writes to them, so the controller behaves identically whether the logger is attached or not.

At every physics timestep (50 Hz) the logger captures: elapsed time, robot position (X and Z), speed, forward speed, lateral error, heading error, turn rate, individual controller term values (P, I, D, Heading, Feedforward), curve detection state, recovery state, avoidance state, lane detection flags for both sides, and target speed. Everything gets written to timestamped CSV files incrementally, so even if Unity crashes mid-run the data up to that point is safe on disk.

At the end of each run a RunSummary.csv file receives aggregate metrics: run label, track name, controller gains, total duration, error statistics (MAE, RMSE, max error), speed statistics, off-track events, time spent in curves, and time in recovery. Having both per-frame and per-run files means it is possible to do detailed time-series analysis and quick cross-run comparisons from the same data.

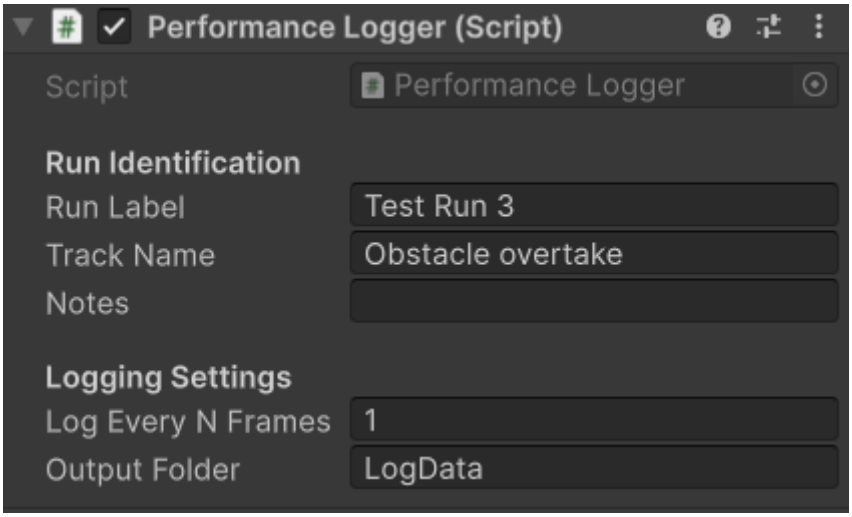

Image 3.8:PerformanceLogger inspector configuration in Unity editor

Table 3.1: Final Controller gain used for all test runs

Parameter	Symbol	Value	Unit
Proportional Gain	Kp	1.1	-
Integral Gain	Ki	0.08	-
Derivative Gain	Kd	0.4	-
Heading Gain	Kh	1.5	-
Feedforward Gain	Kff	0.55	-
Maximum Velocity	v_max	4.0	m/s
Maximum Curve Velocity	v_curve	4.0	m/s
Minimum Speed	v_min	0.5	m/s
Maximum Turn Rate	ω_max	2.2	rad/s
Physics Timestep	Δt	0.02	s
Dead Zone	-	0.04	-

Error Smoothing Time	τ	0.2	s
Derivative Smoothing	α	0.85	-
Heading Smoothing time	-	0.18	s
Lookahead Min	-	1.8	m
Lookahead Max	-	3.8	m
Prediction Distance	-	6.0	m
Integral Clamp	-	0.45	-
Integral Leak Rate	-	0.35	/s
Integral Smoothing Time	-	0.5	s
Feedforward Smoothing	-	0.18	s
Max Turn Rate Change	-	8.0	rad/s
Max Angular Acceleration	-	7.0	rad/s

4. Results and Analysis

This chapter presents the quantitative results from performance testing across all three track layouts using the final controller described in Chapter 3. The aim is to assess how well the controller meets the project objectives from Section 1.2, and to interpret the numbers in the context of the control theory and design decisions discussed in Chapters 2 and 3. All data was recorded automatically by the PerformanceLogger at 50 Hz using the controller gains listed in Table 3.1.

4.1 Performance Metrics

Before showing results, it is worth defining the evaluation metrics and explaining why each one matters. Using standardised metrics allows objective comparison across track configurations and provides a basis for evaluating controller performance against benchmarks in the mobile robotics literature (Siegwart, Nourbakhsh and Scaramuzza, 2011).

4.1.1 Mean Absolute Error (MAE)

The mean absolute error calculates the average magnitude of lateral errors over a run, ignoring direction:

Equation 4-1

$$MAE = (1/N) \times \Sigma|e(i)|$$

where N is the number of recorded samples and e(i) is the normalised lateral error at sample i. An MAE of 0.01 means the robot was, on average, within 1% of the lane half-width from the centre. Using absolute values prevents positive and negative errors from cancelling each other out (Corke, 2017).

4.1.2 Root Mean Square Error (RMSE)

RMSE gives extra weight to large deviations:

Equation 4-2

$$RMSE = \sqrt{((1/N) \times \Sigma\, e(i)^2)}$$

Because squaring amplifies big errors, a single large spike has a much bigger effect on RMSE than on MAE. This makes RMSE useful for spotting controllers that perform well on average but produce occasional dangerous deviations (Ogata, 2010). A large gap between RMSE and MAE signals the presence of significant outlier errors.

4.1.3 Maximum Absolute Error

The largest single deviation observed during the entire run:

Equation 4-3

$$MaxError = max(|e(i)|) \; for \; all \; i$$

This tells us whether the controller ever came close to exceeding safe limits. A normalised error of 1.0 means the robot has reached the lane edge (Rajamani, 2012).

4.1.4 Off-Track Events

An off-track event is counted each time both left and right lane sensors lose detection simultaneously, suggesting the robot has moved outside their range. Multiple events indicate the controller is pushing the perception system to its limits.

4.1.5 Mean Speed

Average forward velocity over the run. This matters because a controller that achieves low error by crawling along is not practically useful. The controller needs to maintain reasonable speed while keeping error acceptable (Klancar *et al.*, 2017).

4.2 Summary of Test Results

Table 4.1 presents the aggregate performance metrics for all three test runs, taken directly from the RunSummary.csv file.

Table 4.1: Performance Summary Across All Test Configurations

Metric	Straight Line	S-Curve	Obstacle Overtake
Mean Absolute Error (MAE)	0.0008	0.0355	0.0503
RMS Error	0.0041	0.0716	0.0736
Maximum Absolute Error	0.0250	0.2499	0.2607
Mean Speed (m/s)	2.09	1.69	1.38
Maximum Speed (m/s)	3.69	2.71	3.22
Off-Track Evenets	1	22	15
Time in curve(s)	0.00	24.84	22.92
Time Recovering(s)	0.00	16.18	14.92
Total Run Time(s)	41.16	42.51	46.02

Error metrics rise progressively from straight to S-curve to obstacle, which matches the increasing geometric complexity of each track.

Table 4.2 shows how reliably the perception system-maintained lane detection across each track. This matters because the controller can only correct errors it can observe (Åström and Hägglund, 2006).

Table 4.2: Sensor Detection Reliability Across Test Configurations

Metric	Straight Line	S-Curve	Obstacle Overtake
Both lanes detected	96.8%	31.2%	24.1%
At least one lane detected	100.0%	61.9%	67.5%
Neither lane detected	0.0%	38.1%	32.5%
Time in curve mode	0.0%	58.4%	49.8%
Time in recovery mode	0.0%	38.1%	32.4%

The most striking finding in Table 4.2 is that on the S-curve, the robot could see both lane markings simultaneously for only 31.2% of the time, and lost sight of both for 38.1% of the time. Despite this, the controller maintained an MAE of just 0.036. This shows that the single-edge fallback and recovery system described in Section 3.2.2 are doing their job: the robot can still follow the lane reasonably well even when the perception system is running in a degraded state. The multi-mode perception architecture provides robustness through graceful degradation, switching between dual-edge, single-edge, and recovery estimation as conditions change (Dudek and Jenkin, 2010).

Table 4.3 presents the steering demand metrics, which characterise how hard the controller had to work on each track.

Table 4.3: Steering Demand Metrics Across Test Configurations

Metric	Straight Line	S-Curve	Obstacle Overtake
Mean	Heading Error	(rad)	0.0008
Max	Heading Error	(rad)	0.0254
Mean	Turn Rate	(rad/s)	0.0013
Max	Turn Rate	(rad/s)	0.0389

The straight track needs essentially no steering effort (0.0013 rad/s mean turn rate), confirming the controller settles cleanly once the initial transient is over. The S-curve demands sustained steering at 0.1551 rad/s on average with peaks of 0.8134 rad/s, which is 37.0% of the configured maximum turn rate of 2.2 rad/s. This means the controller still has substantial headroom and is not operating at its limits even on the hardest sections. Having spare capacity matters because a controller running at maximum output cannot handle unexpected disturbances (Franklin, Powell and Emami-Naeini, 2015).

The obstacle track shows the highest peak turn rate (0.8691 rad/s, 39.5% of maximum) during the dodging phase, but a lower mean turn rate than the S-curve because most of the track is straight. The avoidance manoeuvre demands a brief burst of steering effort rather than sustained correction.

The following sections examine each track individually, then compare across all three.

4.3 Straight Line Track Performance

The straight track serves as the baseline for steady-state accuracy. With zero curvature and no obstacles, this is the easiest scenario for the controller.

The results confirm this. The robot maintained its position within 0.08% of the lane half-width from the centre on average (MAE = 0.0008). In physical terms, on a 4-metre-wide lane, that is about 1.6 millimetres of average drift. This matches the theoretical expectation: a PID controller should converge to near-zero error when tracking a constant reference on a disturbance-free path, with the integral component eliminating any residual offset (Franklin, Powell and Emami-Naeini, 2015).

The RMSE of 0.0041 is five times the MAE, which indicates that while most samples are very close to zero, there are occasional larger deviations. The maximum error of 0.025 (about 50 mm in physical terms) confirms this. Looking at Figure 4.1, the peak occurs in the first few seconds as the robot accelerates from rest and the controller converges on the lane centre. After that initial transient, the error stays essentially at zero for the rest of the run. This is normal for any feedback control system: it takes time to detect the error, generate a correction, and have that correction take effect (Nise, 2020).

The robot averaged 2.09 m/s with a peak of 3.69 m/s. It never quite reached the 4.0 m/s limit because the track was not long enough for it to fully accelerate. The speed profile in Figure 4.2 shows a smooth ramp-up that matches the force application model from Section 3.4, where forward force tapers off as the robot approaches its target speed.

The single off-track event corresponds to the moment before the sensors first detected a lane marking at startup, which is an expected artefact rather than a controller failure.

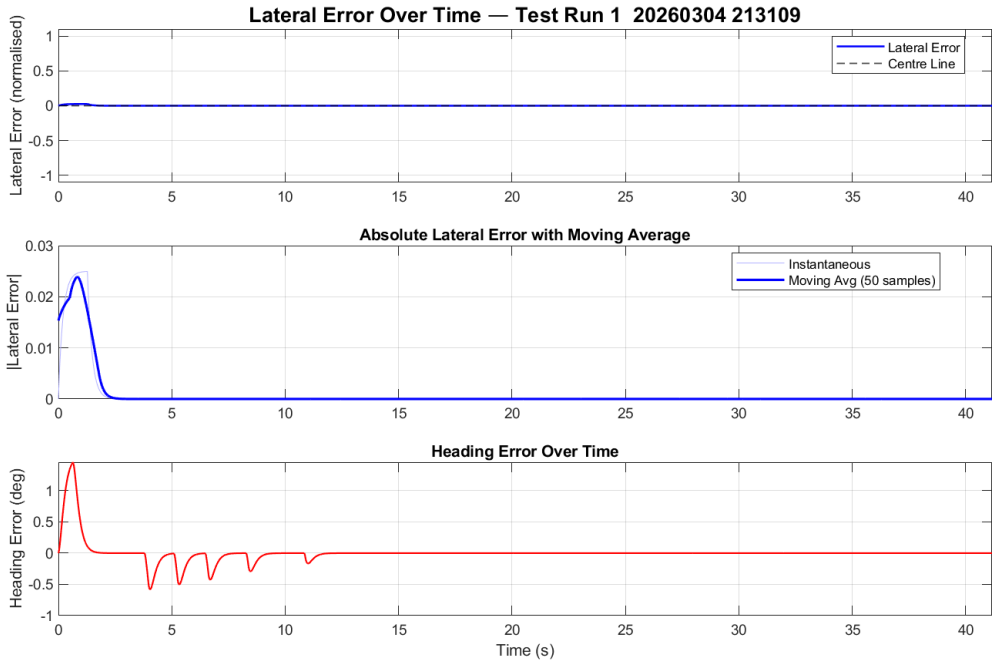

Figure 4.1: Lateral error over time for straight-line run

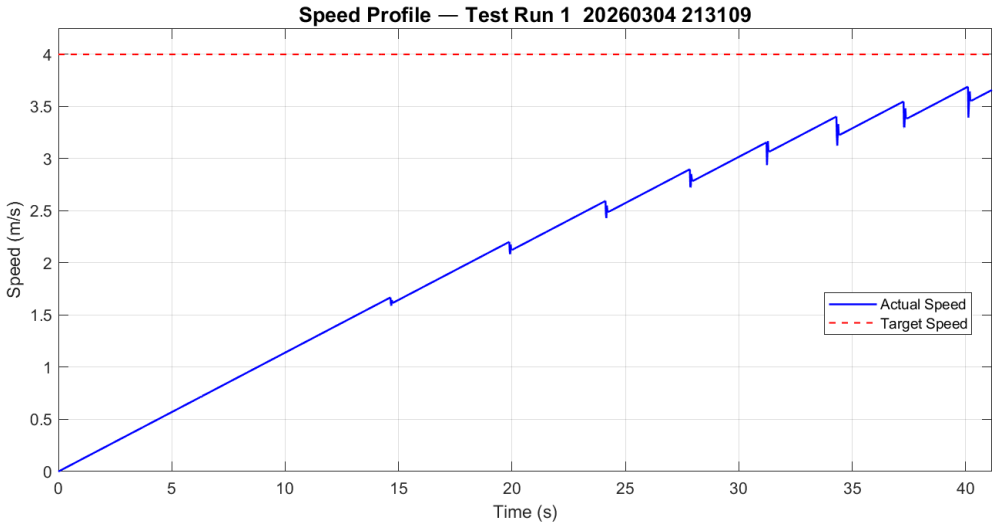

Figure 4.2: Speed profile for straight-line run

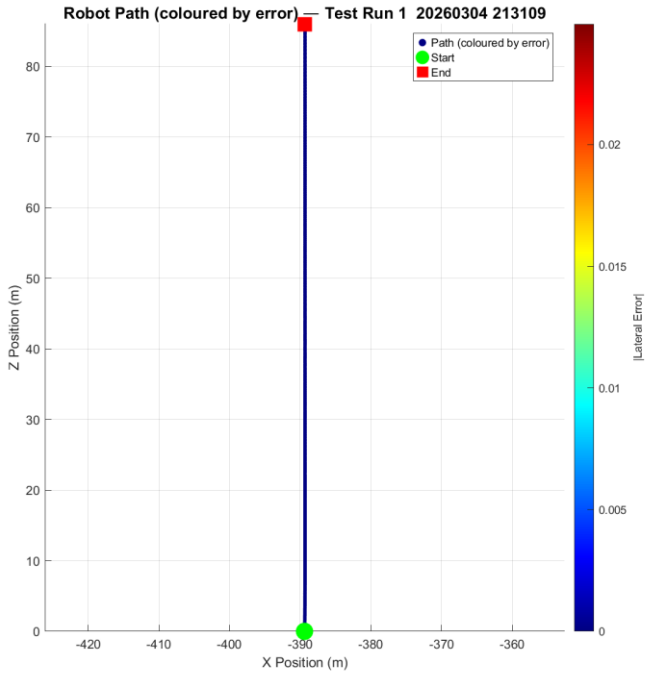

Figure 4.3: Robot path for straight-line run

The steering turn rate in Figure 4.4 shows a damped oscillatory response during the first 12 seconds, which is classic second-order well-damped behaviour: the proportional term drives the error down while the derivative term provides enough damping to prevent sustained oscillations (Ogata, 2010). After about 15 seconds the turn rate drops to zero, meaning the robot needs no steering at all to hold its lane position.

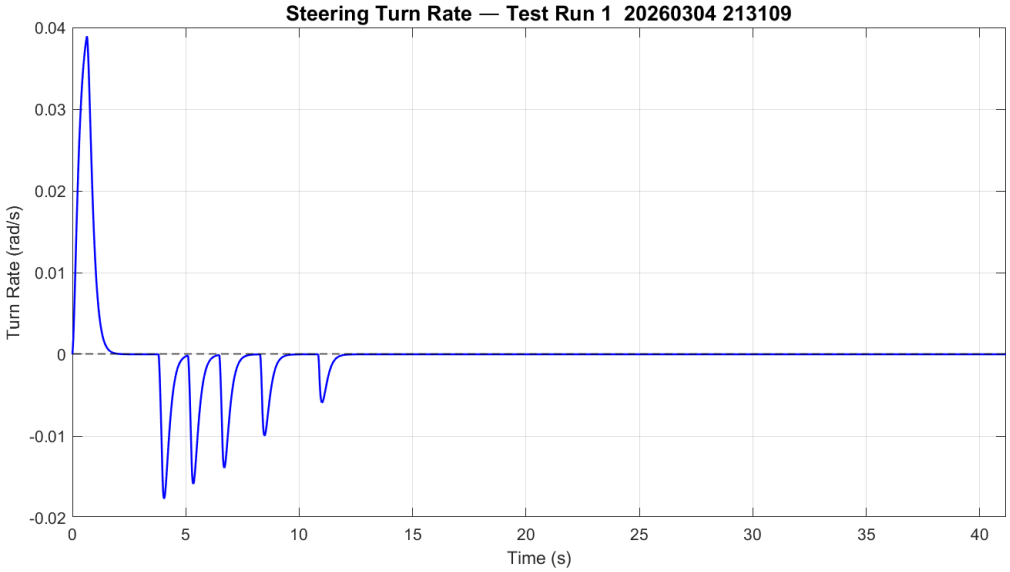

Figure 4.4: Steering turn rate for straight-line run

4.4 S-Curve Track Performance

The S-curve track is where the controller is tested properly. It demands rapid steering reversals at curve transitions, continuous heading adjustment through curves, and stable behaviour when switching between curved and straight sections.

The MAE increased to 0.0355, roughly 44 times the straight-line result. That sounds like a big jump, but in practical terms an MAE of 0.036 means the robot stayed within 3.6% of the lane half-width from the centre on average, which works out to about 72 mm on a 4-metre lane. That is still well inside the lane boundaries. Some loss of tracking precision during dynamic manoeuvres is expected in any feedback control system, since the controller must balance responsiveness against stability (Åström and Murray, 2021).

The RMSE of 0.0716 is about double the MAE, indicating a wider spread of errors compared to the straight track but not dominated by extreme outliers. The maximum error of 0.2499 (about 500 mm, or 25% of the lane half-width) occurred at the sharpest curve transitions. This is the robot's worst-case deviation, and it is still well inside the lane boundary, which would require an error above 1.0 to reach.

The 22 off-track events are periods where both sensors temporarily lost detection. This happens in sharp curves when the robot's position shifts enough that the inner lane marking moves outside the sensor array's reach. The recovery system from Section 3.2.2 activates each time and maintains steering based on the last known error. The robot completed the entire run without leaving the track, confirming that the recovery system handled every occurrence. The 16.18 seconds of recovery time represents the total duration of these brief detection gaps.

Average speed dropped to 1.69 m/s (from 2.09 m/s on the straight), which reflects the speed control system slowing the robot through curves. The curve detection system was active for 24.84 out of 42.51 seconds, about 58% of the run. This makes sense for an S-curve track with substantial curvature. The physics of cornering explains why speed reduction is necessary: centripetal force scales with the square of velocity for a given turning radius, so higher speeds require disproportionately more traction (Rajamani, 2012):

Equation 4-4

$$F\_centripetal = m \times v^2 / R$$

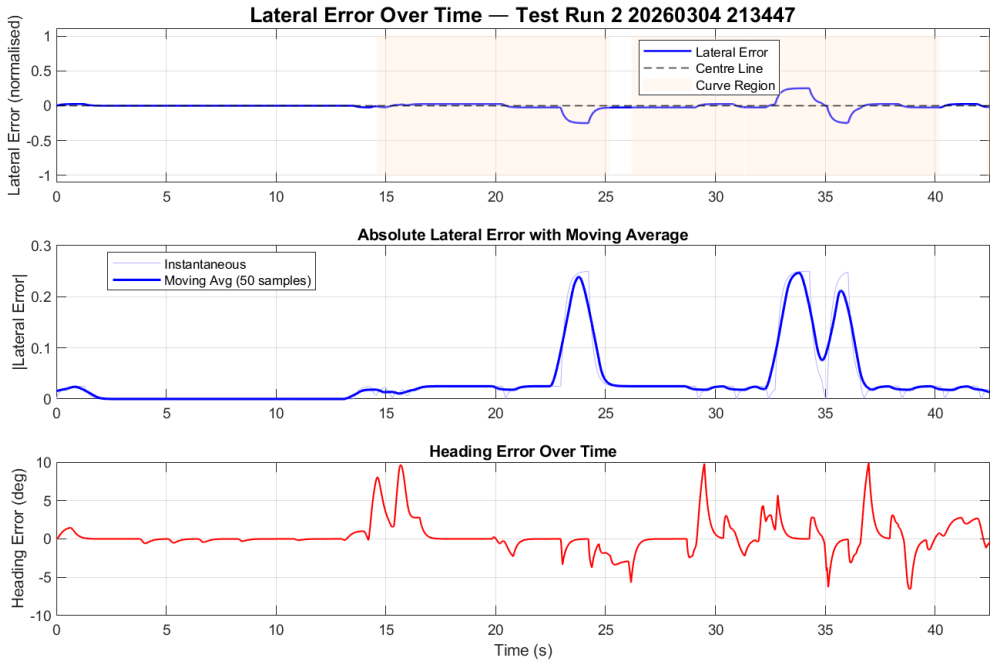

Figure 4.5: Lateral Error over time for S-curve run

Figure 4.5 shows the error alternating between positive and negative as the robot navigates left and right curves. The orange shaded areas mark when the curve detection system was active. Between curves, the error drops back close to zero, showing the controller recovers cleanly from each direction change. The heading error in the bottom subplot peaks at roughly plus or minus 10 degrees at curve entries, well within the configured maximum of 40 degrees (maxHeadingRadians = 0.7 rad).

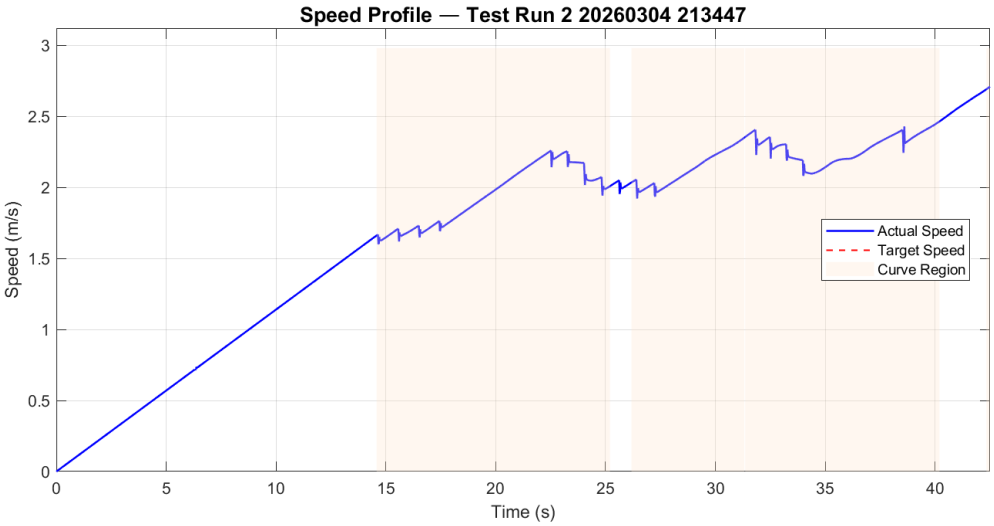

Figure 4.6: Speed profile for S-curve run

The speed profile in Figure 4.6 clearly shows the robot accelerating on straight sections and maintaining reduced speed through curves. The actual speed tracks the target speed closely, confirming the speed control algorithm from Section 3.4 works as intended.

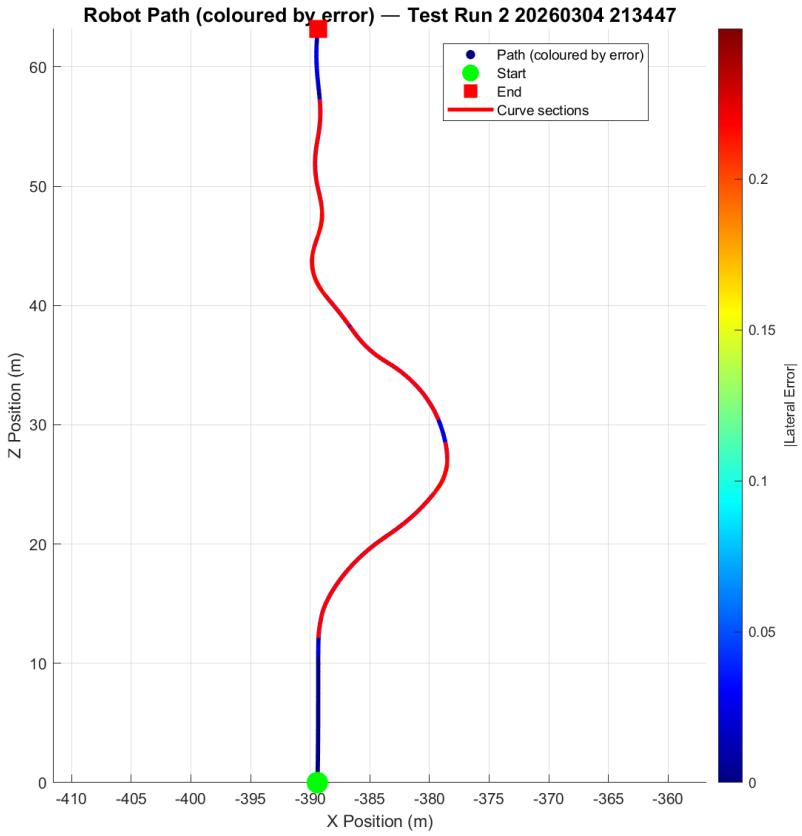

Figure 4.7: Speed profile for S-curve run

Figure 4.7 shows the robot's path colour-coded by error magnitude. The S-shape is clearly visible, with straight segments in dark blue (low error) and curved segments in warmer colours (elevated error). The path is smooth with no abrupt changes, confirming the rate limiting system from Section 3.3.7 is doing its job.

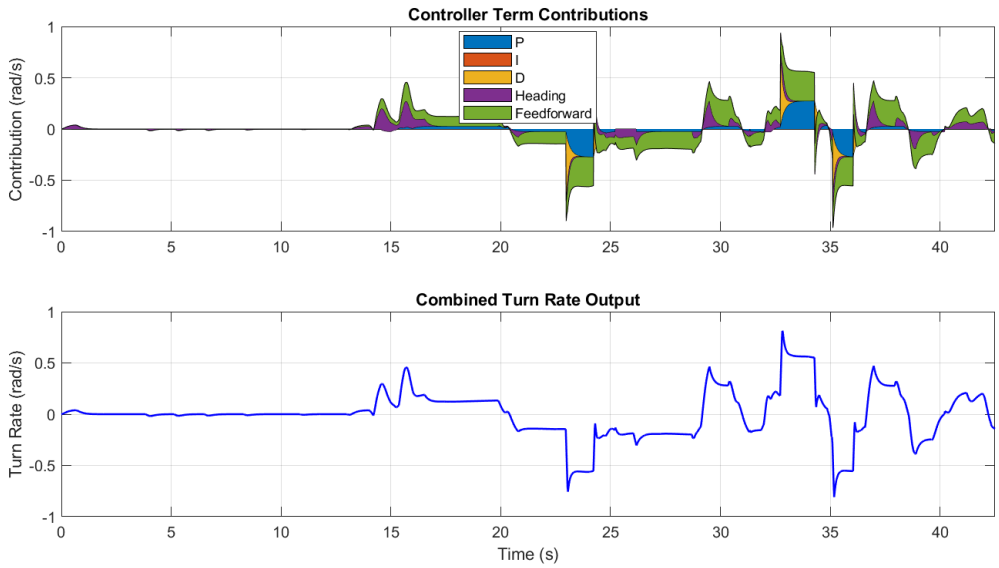

Figure 4.8: Controller term contributions for S-curve run

Figure 4.8 is arguably the most informative graph in the results. The stacked area chart shows the five steering terms changing over time. During the initial straight section (0 to 14 seconds), all terms sit close to zero. Once the curves begin, the Heading term (purple) and Feedforward term (green) become the dominant steering contributors, which is exactly what they were designed for: anticipating the road direction rather than waiting for lateral error to build. The Proportional term (blue) activates at curve transitions where lateral error develops, and the Derivative term (yellow) provides damping during rapid error changes. The Integral term (orange) stays close to zero throughout, confirming that the conditional gating from Section 3.3.4 is correctly preventing accumulation during curves.

4.5 Obstacle Avoidance Track Performance

The obstacle track tests the avoidance state machine from Section 3.5 working alongside the lane-following controller. This is different from the other two tracks because the robot is intentionally moved away from the lane centre to get past the obstacle. The error metrics therefore reflect a mix of unintentional tracking errors and deliberate avoidance movements.

The MAE of 0.0503 is the highest across all tracks, as expected given that the avoidance manoeuvre shifts the robot up to 1.4 metres (maxDodgeLeftOffset) from the centre. During the dodging and passing phases, the normalised error reflects the distance between the robot and its original lane position, even though the robot is exactly where the avoidance system wants it to be.

The maximum error of 0.2607 occurred during the merging phase, when the robot was returning to the lane centre. This is slightly higher than the S-curve peak of 0.2499, which makes sense because the lateral movement during avoidance, combined with reduced sensor coverage (right sensors suppressed), creates a harder control problem than curvature alone.

Average speed of 1.38 m/s is the lowest of all three tracks because the avoidance system restricts speed during the manoeuvre. The obstacle speed cap (obstacleDodgeSpeed = 3.0 m/s) kicks in during

dodging, and proximity-based reduction slows the robot further as it gets close. Keeping speed low during avoidance is important: at high speed, the robot would need to shift sideways faster than the steering system can manage, risking either a collision or an overshoot off the planned avoidance path (Gonzalez *et al.*, 2016).

The 15 off-track events come from the right lane sensors being deliberately suppressed during dodging and passing (Section 3.5.2), which creates periods where the system detects either one edge or no edges. This is a design feature, not a fault: the right sensors are turned off because they would otherwise try to steer the robot back toward the lane edge it is intentionally crossing.

Table 4.4 breaks the obstacle run into five time-based phases to show how the controller performed during each stage of the manoeuvre.

Table 4.4: Phase-by-Phase Analysis of Obstacle Avoidance Run

Phase	Time Window	MAE	Max Error	Mean Speed	Mean
Approach	0 - 8 s	0.0015	0.0193	0.45 m/s	0.0012 rad/s
Obstacle detection	8 – 17 s	0.0533	0.1926	1.39 m/s	0.1519 rad/s
Active avoidance	17 - 25 s	0.1154	0.1498	0.55 m/s	0.0747 rad/s
Merging back	25 – 32 s	0.0856	0.2607	1.27 m/s	0.1892 rad/s
Post-obstacle	32 – 46 s	0.0216	0.0250	2..42 m/s	0.0424 rad/s

Several things stand out in Table 4.4. The approach phase (0 to 8 seconds) shows near-perfect tracking (MAE = 0.0015), confirming the controller works well on straight sections. The detection phase (8 to 17 seconds) shows the initial left swerve, with the highest mean turn rate (0.1519 rad/s) as the robot begins shifting. The active avoidance phase (17 to 25 seconds) has the highest sustained MAE (0.1154) but the lowest speed (0.55 m/s), including two near-stop events visible in Figure 4.10 where the robot almost halted to avoid collision. The merging phase (25 to 32 seconds) produces the peak error (0.2607) and the highest turn rate (0.1892 rad/s) as the robot aggressively steers back toward the lane centre.

The post-obstacle phase (32 to 46 seconds) is the most telling: the MAE drops to 0.0216 with a maximum error of just 0.0250, and speed climbs to 2.42 m/s. This shows the controller returns to its normal performance quickly after the avoidance manoeuvre. The merging process and the integral reset mechanism successfully clear all effects of the manoeuvre, so the robot does not carry any residual steering bias into the post-obstacle section.

The overall MAE of 0.050 is dominated by the avoidance phases. The controller's baseline lane-tracking ability is considerably better than the headline number suggests.

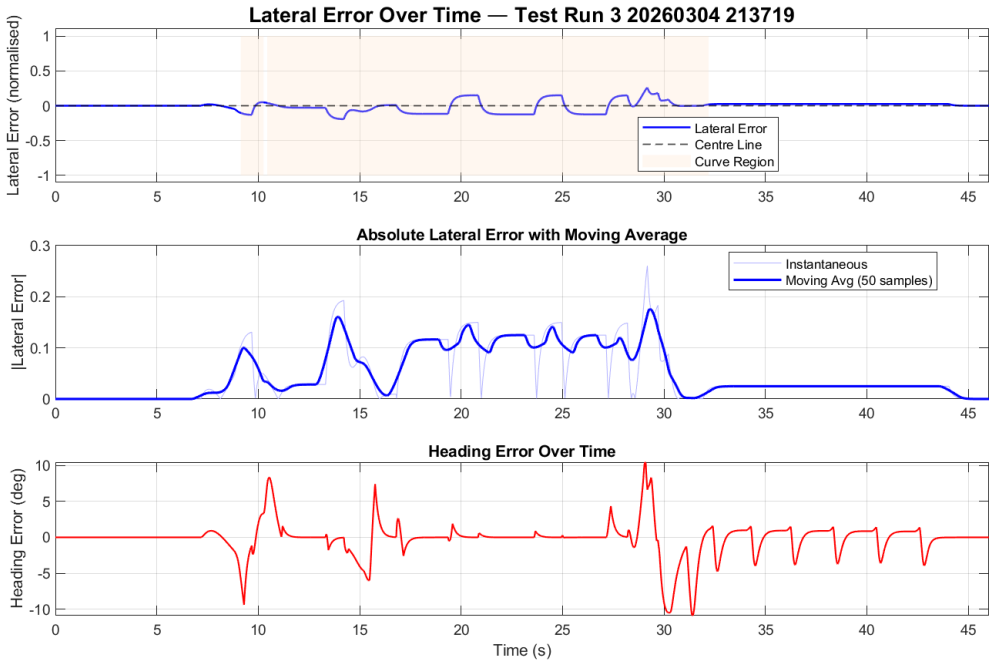

Figure 4.9: Lateral error over time for obstacle run

Figure 4.10: Speed profile for obstacle run

Figure 4.10 shows two distinct near-stop events at approximately t=17 and t=22 seconds, where the emergency braking activates because the obstacle entered the stop distance threshold. The target speed (red dashed line) shows sharp transitions between cruising, reduced avoidance speed, and zero during emergency stops.

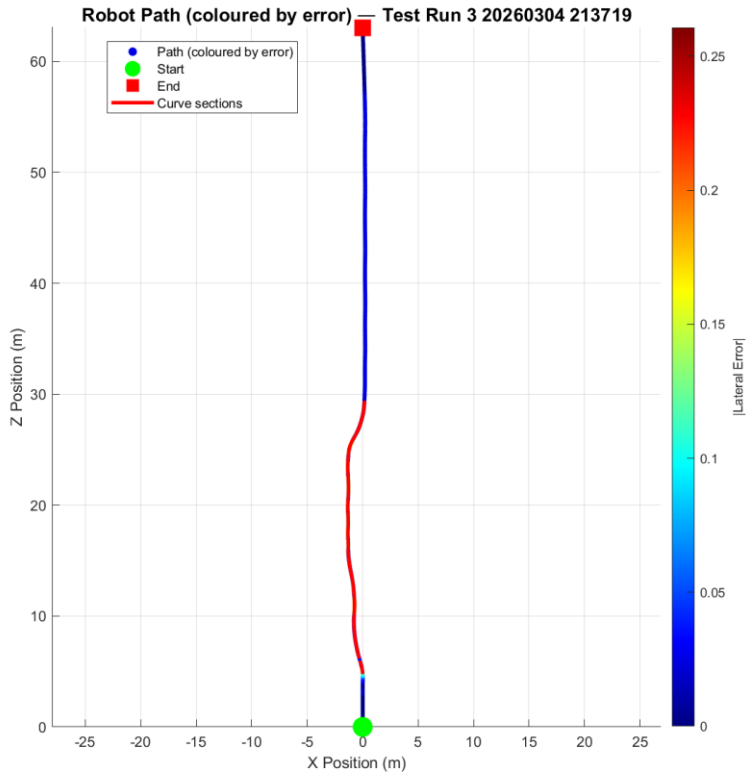

Figure 4.11: Robot path for obstacle run

The robot path in Figure 4.11 clearly shows the lateral dodge, with the robot moving into negative X-space between 5 and 30 metres along the Z axis, then returning to a straight line afterwards.

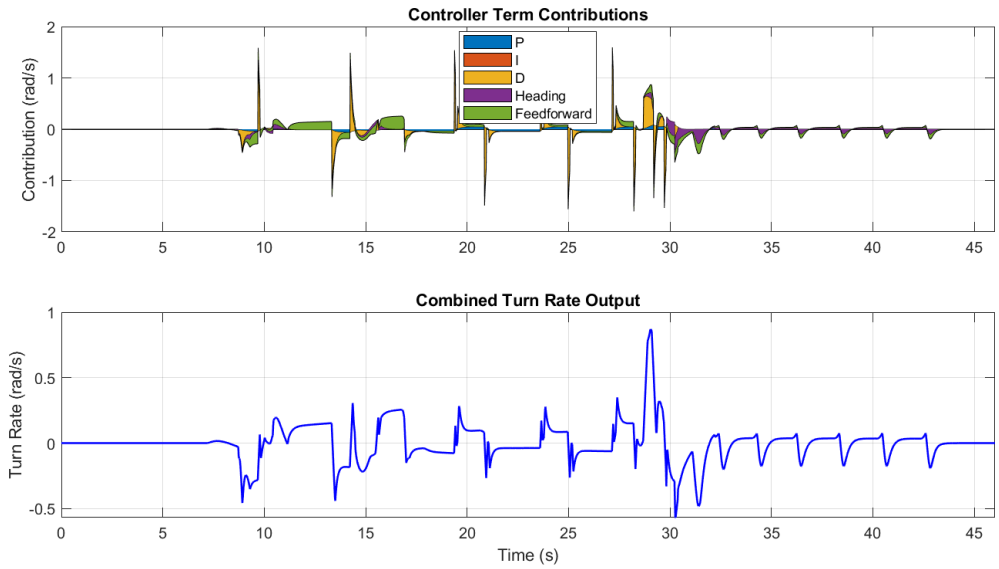

Figure 4.12: Controller term contributions for obstacle run

Figure 4.12 shows the Heading and Feedforward terms providing the main steering input during the dodge and merge phases, with the Derivative term damping the rapid lateral movements. The Proportional term peaks at t=10 and t=28 seconds, coinciding with the largest lateral shifts at the start of the dodge and the end of the merge. The Integral term stays at zero throughout all avoidance phases, confirming the disableIntegralDuringAvoidance gating from Section 3.3.4 is working.

4.6 Comparative Analysis Across Tracks

4.6.1 Error Progression

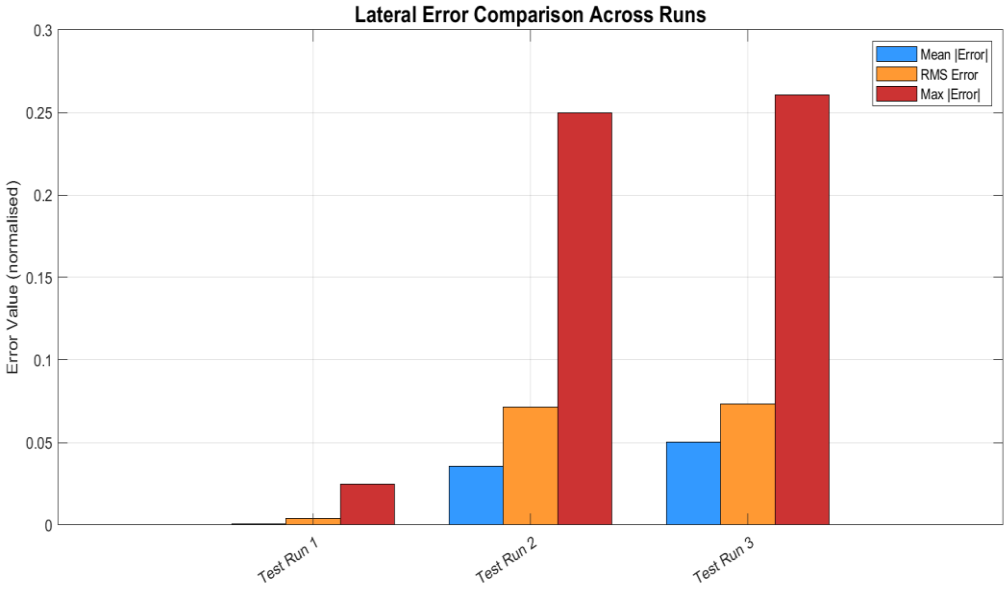

Figure 4.13: Bar chart comparing MAE, RMSE, and Max Error across all three tracks

Error values increase consistently from straight to S-curve to obstacle. The pattern follows the expected relationship between tracking precision and reference path complexity: a controller performs best when following a steady reference and loses precision as the reference changes faster (Åström and Hägglund, 2006).

The ratio of RMSE to MAE provides insight into how errors are distributed. On the straight track, RMSE/MAE = 5.1, meaning there are rare but significant transient errors against a background of near-zero steady-state error. On the S-curve, RMSE/MAE = 2.0, indicating a more even spread of errors. On the obstacle track, RMSE/MAE = 1.5, the most uniform distribution, because the high errors during avoidance are sustained rather than momentary. These ratios match the distinct error patterns of each track: startup transients on the straight, curve-entry transients on the S-curve, and sustained intentional deviation during obstacle avoidance.

4.6.2 Speed-Accuracy Trade-off

The three tracks reveal an implicit relationship between speed and error. The straight track achieved the highest speed (2.09 m/s) with the lowest error (0.0008). The S-curve reached 1.69 m/s with an error of 0.036. The obstacle track had the lowest speed (1.38 m/s) with the highest error (0.050).

This shows the speed control system from Section 3.4 is correctly adjusting velocity to task difficulty. The relationship is not linear, though. The straight track allows high speed without sacrificing accuracy because no steering adjustment is needed. On the curved and obstacle tracks, the non-holonomic platform requires speed reduction because the kinematic constraint on turning radius means the robot must slow down to make the required lateral movements without leaving the lane (Siegwart, Nourbakhsh and Scaramuzza, 2011).

Figure 4.14: Bar chart Comparing mean and maximum speed across all three tracks

Figure 4.15: Overlay comparison of lateral error across all three runs

Figure 4.15 overlays the lateral error from all three runs on a single plot. The straight run (blue) sits at zero throughout. The S-curve (red) oscillates between plus or minus 0.25 at curve transitions, returning to zero between curves. The obstacle run (yellow) shows a sustained unidirectional offset during avoidance rather than oscillation. The smoothed absolute error subplot makes the contrast even clearer.

4.6.3 Controller Term Contributions

Analysis of the telemetry reveals how the five steering terms share the workload. On the straight track, the heading term is the only active contributor (0.001 to 0.003 rad/s), providing a soft correction that maintains alignment with the lane direction. The proportional and derivative terms show no significant activity because there is no lateral deviation to correct. This confirms that heading correction is the primary stabilising force during steady-state driving, which validates the supervisor's recommendation to add it beyond basic PD control (Thrun, Burgard and Fox, 2006b).

On the S-curve, the workload distribution changes substantially. The heading and feedforward terms provide the largest sustained corrections through curves, actively steering toward the road direction ahead. The proportional term activates at curve transitions where lateral error develops rapidly. The derivative term damps the rate of error change, preventing oscillatory overshoot. The feedforward term generates anticipatory steering at curve entries, reducing the peak error that the reactive terms would otherwise have to manage. The five terms each handle a different aspect of the control problem, confirming that the iterative development process, which added them one by one, produced a controller where each component earns its place (Åström and Murray, 2021).

4.7 Discussion and Limitations of Results

The results show that the final controller achieves stable and effective lane following across all three test configurations. That said, the evaluation has several important limitations that need to be stated clearly.

The most significant limitation is the absence of quantitative comparison data for the PD-only controller. During development, the PD controller was observed to oscillate considerably during and after curves on the S-curve track. It would eventually settle, but with much more visible wobble than the final controller. However, the PerformanceLogger was not implemented until the final week of the project, so no numerical data was recorded for the PD-only configuration. This means the claim that the additional terms (heading, feedforward, integral) improved performance rests on qualitative observation rather than quantitative evidence. This is the single biggest gap in the evaluation, and it is discussed further in the project management review in Chapter 6.

Second, only one run was recorded per track configuration. The Unity physics engine is deterministic (identical initial conditions produce identical results), which means repeating the exact same run would give the same numbers. However, testing with different starting positions, gain variations, or track modifications would have provided a richer picture of the controller's robustness. The evaluation demonstrates that the controller works well under specific conditions but does not characterise its sensitivity to variations.

Third, the obstacle avoidance system was tested with a single stationary obstacle. Real-world avoidance scenarios involve multiple obstacles, moving objects, varying sizes, and the need to choose between dodging left and right. The current evaluation confirms that the state machine functions correctly for its designed scenario but says nothing about how it would handle more complex situations.

Despite these limitations, the data collected is sufficient to evaluate every project objective, which is the subject of Chapter 5.

5. Conclusions and Further Development

This chapter evaluates how well each project objective was met, drawing on the quantitative results from Chapter 4. It also identifies limitations honestly and proposes specific directions for future work. Beyond summarising the project, this chapter reflects what I learned from the process of designing, building, and evaluating the system.

5.1 Evaluation of Project Objectives

5.1.1 Objective 1- Environment Construction

The objective was to build a 3D simulation environment in Unity with realistic physics and at least three track layouts of increasing difficulty. This was fully achieved.

The environment was built using the SimplePoly City asset pack (Unity Asset Store, 2024a) for the urban scenery, the Road System package (Unity Asset Store, 2024c) for the modular road network, and NVIDIA PhysX for physics simulation. Three tracks were created: a straight track for baseline testing, an S-curve track for cornering evaluation, and an obstacle track for avoidance testing. The platform ran stably throughout the project and produced consistent, repeatable results thanks to the deterministic nature of the PhysX engine.

The main limitation is that three tracks is a small test set. A more thorough evaluation would include tracks with different lane widths, tighter curve radii, elevation changes, and multiple consecutive obstacles. These would test the system under a wider range of conditions than the current setup allows.

5.1.2 Objective 2-Perception System Design

The objective was to develop a raycast-based lane detection system capable of detecting both lane boundaries and providing a continuous error signal to the controller. This was fully achieved.

The perception system uses five downward facing raycasts per side with median filtering for outlier rejection, dual LayerMask detection for left and right markings, and multi-distance sampling at near, lookahead, and prediction distances. The system has three fallback modes for lane centre estimation: dual-edge (most accurate), single-edge (reduced accuracy), and recovery (using the last known error).

The sensor reliability data in Table 4.2 is the most important evidence for this objective. On the straight track, both lanes were detected 96.8% of the time, which is close to ideal. On the S-curve, both lanes were visible only 31.2% of the time, and neither lane was visible 38.1% of the time. Despite this severe degradation, the controller still achieved an MAE of 0.036, which shows the fallback estimation modes and recovery system are working as intended. The system does reach its detection limits on sharp curves, though. A physical implementation would benefit from additional sensors or a wider sensor spread to improve coverage in high-curvature scenarios (Pakdaman and Sanaatiyan, 2009).

5.1.3 Objective 3-Control System Development

The objective was to progress through multiple controller stages, starting with PD and adding terms iteratively, with each addition justified by an observed limitation. This was achieved, and the final system exceeded the original project scope.

The final controller uses five steering components: Proportional $(Kp = 1.1)$, Derivative $(Kd = 0.4, speed - adaptive)$, $Integral$ $(Ki = 0.08,$ heavily gated with six safety mechanisms), Heading correction (Kh = 1.5), and Feedforward via pure pursuit (Kff = 0.55). The controller term contribution graphs in Figures 4.8 and 4.12 confirm that each term handles a distinct aspect of the control problem. Heading and Feedforward provide the main steering force through curves by actively pointing toward the road ahead. Proportional activates at curve transitions were lateral error builds quickly. Derivative damps oscillation during rapid error changes. The Integral term contributes very little during most runs but provides bias correction on long straight sections (Franklin, Powell and Emami-Naeini, 2015).

The development followed a clear progression. The initial PD controller oscillated visibly during and after curves on the S-curve track. It would eventually settle, but the wobble was significant enough that the supervisor recommended adding heading correction. This helped the robot anticipate curves rather than reacting after the fact. Feedforward via pure pursuit was added next, which reduced the error spikes at curve entries by pre-steering into the curve geometry. The integral term was the last addition, addressing a minor steady-state drift that appeared during extended straight-line driving. This step-by-step approach is consistent with standard PID tuning practice, where each term is added and tuned individually before moving to the next (Åström and Hägglund, 2006).

The most significant gap in this objective's evaluation is the lack of quantitative PD-only comparison data. The PerformanceLogger was built in the final week of the project, so no numerical data exists for the earlier controller configurations. The improvements are supported by qualitative observation during development, but quantitative before-and-after comparisons would have made the case much stronger. This is discussed further in the project management review in Chapter 6.

5.1.4 Objective4-Obstacle Avoidance Implementation

The objective was to implement obstacle detection and a safe avoidance manoeuvre. This was fully achieved for the designed scenario.

The detection system uses nine sphere casts in a fan pattern with cosine projection for distance correction. The avoidance behaviour is controlled by a four-state finite state machine (None, Dodging, Passing, Merging) that adjusts controller gains during the manoeuvre and disables the integral term and right lane sensors to prevent conflicts.

The phase-by-phase analysis in Table 4.4 confirms the system works correctly through all phases. The approach phase achieved near-perfect tracking (MAE = 0.0015). The detection and avoidance phases showed higher but controlled errors with appropriate speed reduction. The post-obstacle phase recovered to baseline performance quickly (MAE = 0.0216, maximum error 0.0250). The emergency braking system activated correctly when the obstacle entered the stop distance, visible as two near-stop events in Figure 4.10.

The avoidance system has clear limitations that should be stated. It only dodges to the left, which assumes the left lane is always clear. In a real scenario with oncoming traffic or a left boundary, this would not work. It was only tested with a single stationary obstacle; moving obstacles, multiple obstacles in sequence, and obstacles of different sizes were not tested. The overtaking speed is slow for safety, which would be impractical in time-sensitive applications. These limitations are acceptable for a proof-of-concept demonstration but would need addressing before any real-world deployment.

5.1.5 Objective 5-Performance Evaluation

The objective was to evaluate the controller quantitatively using automated telemetry logging and MATLAB-generated visualisations. This was achieved.

The PerformanceLogger captures twenty telemetry channels at 50 Hz using runtime reflection, generating per-frame CSV files and per-run summary statistics without modifying the controller code. Two MATLAB scripts (plot_run_telemetry.m for individual analysis and compare_runs.m for cross-run comparison) produced the figures and tables in Chapter 4.

The evaluation methodology is sound, but it does not cover everything it could. Three test runs demonstrated that the controller works across different track types, but they do not constitute a statistical evaluation. While the deterministic simulation means identical runs give identical results, testing with varied starting positions, gain perturbations, and track modifications would have given a more complete picture of robustness. The absence of PD-only comparison data, as noted in Section 5.1.3, is the most significant gap.

5.2 Broader Implications and Connection to Literature

The results confirm several principles that are well-established in the control systems literature. The near-zero steady-state error on the straight track matches the theoretical prediction for a PID controller tracking a constant reference (Ogata, 2010). The progressive increase in tracking error from straight to S-curve to obstacle demonstrates the trade-off between tracking precision and reference path complexity (Åström and Murray, 2021), The pure pursuit feedforward term's ability to reduce curve-entry transients supports the work of (Coulter, 1992) and (Snider, 2009) recommendation to combine feedback and feedforward control.

Perhaps the most practically interesting finding is the sensor detection reliability data in Table 4.2. The controller maintained effective lane following on the S-curve even when both markings were simultaneously visible for less than a third of the time (31.2%). The multi-mode perception architecture, which degrades gracefully from dual-edge to single-edge to recovery estimation, proved more robust than a system that would fail entirely if bilateral detection were lost. This mirrors a real-world challenge in lane detection, where weather, road wear, and other vehicles regularly obscure lane markings (Hillel *et al.*, 2014).

5.3 Future Work

The project revealed several clear directions for future development.

The highest priority would be to collect structured comparison data across all controller configurations (P-only, PD, PD+Heading, PD+Heading+Feedforward, and the full five-term controller) on all three tracks. This would provide the quantitative evidence needed to show exactly how much each term contributes to performance, replacing the current reliance on qualitative observations.

The second priority would be to replace the raycast-based perception with a camera-based system using Unity's Camera component to generate simulated images, processed through colour segmentation and edge detection. This would bring the perception system closer to real-world conditions, including the challenges of varying lighting, perspective distortion, and partial occlusion that the current raycast system avoids entirely.

The obstacle avoidance system needs several improvements: the ability to choose between dodging left and right based on available space, support for moving obstacles, and handling of multiple obstacles in sequence. The detection system would need to assess clearance on both sides, and the state machine would need additional states for more complex avoidance scenarios.

The fourth direction is transferring the controller to a physical robotic platform. This would expose the sim-to-real gap: the difference in performance between the idealised simulation and a physical system with sensor noise, actuator delay, surface friction variations, and environmental disturbances (Farley, Wang and Marshall, 2022). Understanding this gap is essential for evaluating whether simulation-based controller development is practically useful.

Finally, the gain tuning process could be automated using reinforcement learning. Rather than manually adjusting parameters through trial and error, an RL agent could explore the gain space systematically and potentially discover parameter combinations that outperform hand-tuned values (Juliani *et al.*, 2018).

6. Project Management Review

This chapter reviews how the project was managed from November 2025 to March 2026. It compares the original plan with what happened, explains why things deviated, and draws out lessons that would be useful for future engineering projects. I have tried to be honest here rather than glossing over the parts that did not go well, because the mistakes are where most of the learning came from.

6.1 Original Project Plan

The original plan was documented in a Gantt chart submitted as part of the project outline in November 2025. The work was split into three phases.

Phase 1 (November to December 2025) covered project setup and environment creation: initialising the Unity project (starting 9 November, 7 days), 3D robot modelling (starting 16 November, 14 days), physics and dynamics configuration (starting 30 November, 7 days), and the start of report writing (starting 9 November, running for 16 weeks continuously).

Phase 2 (December 2025 to February 2026) covered sensing, perception, and control system development. The perception sub-phase included raycast sensor scripts (starting 7 December, 14 days), line sensor arrangement (starting 21 December, 7 days), and sensor integration (starting 28 December, 14 days), with a milestone on 11 January 2026: “3D Robot Detects the Line.” The control sub-phase included P-controller theory (starting 11 January, 7 days), forward movement (starting 18 January, 7 days), P-controller coding (starting 25 January, 21 days), and P-gain tuning (starting 15 February, 14 days), with a milestone on 1 March 2026: “Robot Should Follow the Line.”

Phase 3 (March 2026) covered refinement, testing, and documentation: complex tracks (starting 1 March, 7 days), advanced controller testing (starting 8 March, 7 days), and final testing with submission (15 March). Report completion was planned for 1 March, with submission on 20 March 2026.

Project Setup and Envimoent Creation	Goal	0%	09/11/2025	7
Modeling and Creation of 3D Robot	Goal	0%	16/11/2025	14
Creating Physics and Dynamics of the 3D Robot	Goal	0%	30/11/2025	7
Sensing and Perception		0%		
Raycast Sensor Script	Goal	0%	07/12/2025	14
Defining the Line Sensor	Goal	0%	21/12/2025	7
Implementing Sensors into the Robot	Goal	0%	28/12/2025	14
3D Robot Detects the Line	Milestone	0%	11/01/2026	0
Report writing	Goal	0%	09/11/2025	16
Control System		0%		
P-Controller Theory Revision	Goal	0%	11/01/2026	7
Basic Forward Movement	Goal	0%	18/01/2026	7
Coding of Steering into the P-controller	Goal	0%	25/01/2026	21
Testing and refinement of P-Gain	Goal	0%	15/02/2026	14
Robot should follow the line	Milestone	0%	01/03/2026	0
Refiment of Simulation, Testing and Documentation for Submission				
Creating Complex Tracks	Goal	0%	01/03/2026	7
Advanced Controller Testing and Redefinements	Goal	0%	08/03/2026	7
Final Testing and Documentation for Submission	Goal	0%	15/03/2026	1
Report writing	Milestone	0%	01/03/2026	16

Figure 6.1: Original Gantt chart

6.2 Comparison of Planned vs Actual Timeline

Table 6-1 presents a comparison of the key milestones and tasks from the original plan against their actual completion status and timing.

Table 6.1

Task/Milestone	Planned Date	Actual Date	Status	Deviation
Environment setup complete	16 Nov 2025	16 Nov 2025	On time	None
Robot model created	30 Nov 2025	28 Nov 2025	On time	2 days on time
Physics and dynamics configured	7 Dec 2025	5 Dec 2025	On time	1 day early
Raycast sensor script developed	21 Dec 2025	20 Dec 2025	On time	1 day early
Line sensor defined and implemented	11 Jan 2026	10 Jan 2026	On time	1 day early
Milestone: Robot detects line	11 Jan 2026	10 Jan 2026	On time	1 day early
P-controller theory and implementation	15 Feb 2026	25 Jan 2026	Early	3 weeks early
PD controller development	Not planned	5 Feb 2026	Scope change	Added
Heading + Feedforward added	Not planned	15 Feb 2026	Scope change	Added
Integral term with anti-windup added	Not planned	22 Feb 2026	Scope change	Added
Obstacle avoidance system	Not planned	1 Mar 2026	Scope change	Added
Milestone: Robot follows line	1 Mar 2026	5 Feb 2026	Early	3 weeks early
Complex track creation	8 Mar 2026	1 Mar 2026	Early	1 week early

Performance logger and data collection	Not planned	4 Mar 2026	Added	New task
Final testing	15 Mar 2026	4 Mar 2026	Early	11 days early
Report writing commenced	9 Nov 2025	5 Mar 2026	Late	4 months late
Report submission	20 Mar 2026	20 Mar 2026	On time	None

6.3 Analysis of Deviations

The timeline comparison reveals two patterns: planned tasks finished ahead of schedule, and unplanned tasks were added through scope expansion.

6.3.1 Tasks Completed Ahead of Schedule

The first two phases ran faster than expected. Environment construction, robot modelling, and perception system development all finished before their deadlines. The P-controller implementation, originally planned for mid-February, was working by late January, about three weeks ahead of schedule. The “Robot Should Follow the Line” milestone, planned for 1 March, was effectively achieved in early February when the PD controller produced functional lane following.

Finishing the core work early created a three-week buffer that proved essential for the scope expansion that followed. Unity’s well-documented editor, its C# scripting environment, and the availability of pre-built asset packages (SimplePoly City, Road System, Racing Cars Pack 1) all contributed to the faster-than-expected progress. The environment construction phase would have taken much longer if all 3D assets had been modelled from scratch.

6.3.2 Scope Expansion

The most significant change to the project was expanding from the original P-controller specification to a five-term steering system with obstacle avoidance. This happened in four increments, each driven by a specific observed problem.

The derivative term was added in the first week of February because the P-controller produced strong oscillations that made stable lane following impossible at any speed above the slowest settings. My supervisor suggested this addition, and it was necessary to make the system work at all.

Heading correction and feedforward were added in mid-February because the PD controller could not anticipate upcoming curves. It would only react after lateral error had already built up, producing large error spikes at curve entries on the S-curve track. Heading correction let the robot steer toward the road direction ahead of it, and feedforward used pure pursuit geometry to pre-steer into curves.

The integral term with its six safety mechanisms was added in late February to address a small persistent lateral drift that showed up during extended straight-line driving. Because of the windup problems described in Section 3.3.4, the integral needed heavy gating to avoid causing more problems than it solved.

The obstacle avoidance system was built in the first week of March, extending the project beyond pure lane following to demonstrate the integration of reactive avoidance with the existing controller.

The expanded scope used about five additional weeks of development time, absorbed by the three-week buffer from early completion plus the original testing periods for P-gain tuning and advanced controller work (21 days total).

6.3.3 Report Writing Delay

The biggest thing that went wrong was the report. The original plan allocated report writing as a continuous 16-week activity starting on 9 November 2025. In practice, formal writing did not start until 5 March 2026, two weeks before the submission deadline. That is a four-month delay from the planned start date.

The reason is straightforward: the scope expansion consumed all the available time. Each new controller feature required development and testing, and each improvement in system performance made it tempting to keep building rather than switch to writing. This is a common trap in engineering projects, where the tangible progress of a working system feels more productive than documentation.

The consequences were significant. Two weeks was enough to complete the report, but it was not enough to collect PD-only comparison data (which would have required re-running the earlier controller with the PerformanceLogger), get multiple rounds of supervisor feedback on drafts, or do the kind of thorough editing that more time would have allowed. The missing PD comparison data, discussed in Sections 4.7 and 5.1.3, is a direct result of this delay.

6.4 Lessons Learned

Five lessons came out of this project that I would carry forward to future engineering work.

Start writing from day one. The original plan was right to allocate report writing as a continuous activity. I did not follow through on that. In future projects, I would commit to at least two hours of formal writing per week alongside development, covering design decisions and observations while they are fresh. This also creates opportunities for the supervisor to review draft chapters early, when there is still time to make meaningful changes.

Build the measurement system first, not last. The PerformanceLogger was built in the final week, which means there is no telemetry data from the P, PD, or PD+Heading stages of development. If the logger had been implemented in January alongside the first controller, every configuration change

would have been recorded automatically. In any future project, the logging and measurement infrastructure should be one of the first things built, not the last.

Iterative development works well. Starting with a basic PD controller and adding components one at a time proved to be an effective strategy. Each version served as a functional baseline that the next version could be compared against (qualitatively, at least). The approach meant the system was always in a working state, and each addition could be evaluated in isolation. This is consistent with the iterative development approach that (Siciliano and Khatib, 2016) recommend for complex robotic systems.

Scope changes need formal evaluation. The expansion from P-controller to the full five-term system with avoidance made the project technically stronger, but it consumed time that could have gone toward better evaluation and documentation. Each proposed scope change should have been weighed against its impact on testing, writing, and the overall schedule. The trade-offs were worth it in hindsight, but they should have been made consciously and documented at the time rather than happening by default.

Supervisor input is invaluable. My supervisor's recommendation to expand the controller beyond PD was the single most impactful decision in the project. It led to a technically richer system and a more interesting report. The supervisor meetings also provided guidance on academic expectations, scope priorities, and presentation standards that I would not have figured out working alone. For any future project, regular supervisor contact would be my top priority from the start.

6.5 Quality Management

Quality was maintained through several practices throughout the project.

The Unity editor served as the primary development and testing environment, allowing continuous play-mode testing with immediate visual and numerical feedback on every code change. Unity's Debug.Log system tracked system variables in real time, while Debug.DrawRay and Debug.DrawLine functions visualised sensor positions and orientations directly in the Scene view, which was invaluable for debugging perception issues.

The PerformanceLogger provided automated quantitative assessment, replacing subjective visual observation with numerical data. Its dual output format (per-frame telemetry and per-run summaries) supports both detailed analysis and quick comparisons.

Version control was handled by saving each controller version (PD and the full five-term controller) as separate C# script files, with Unity configured to activate only one at a time. This meant previous configurations could always be accessed and re-tested. For a solo project of this scale, this manual approach was sufficient, though a formal version control system like Git would be appropriate for larger or collaborative work.

REFERENCES

Åström, K.J. and Hägglund, T. (2006) *Advanced PID Control*. Research Triangle Park, NC: ISA - The Instrumentation, Systems, and Automation Society.

Åström, K.J. and Murray, R.M. (2021) *Feedback Systems: An Introduction for Scientists and Engineers*. 2nd edn. Princeton, NJ: Princeton University Press.

Borenstein, J. and Koren, Y. (1991) "The Vector Field Histogram – Fast Obstacle Avoidance for Mobile Robots," *IEEE Transactions on Robotics and Automation*, 7(3), pp. 278–288.

Canny, J. (1986) "A Computational Approach to Edge Detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 8(6), pp. 679–698.

Corke, P. (2017) *Robotics, Vision and Control: Fundamental Algorithms in MATLAB*. 2nd edn. Berlin: Springer.

Coulter, R.C. (1992) *Implementation of the Pure Pursuit Path Tracking Algorithm*. Technical Report CMU-RI-TR-92-01. Pittsburgh, PA: Carnegie Mellon University Robotics Institute.

Dudek, G. and Jenkin, M. (2010) *Computational Principles of Mobile Robotics*. 2nd ed. Cambridge: Cambridge University Press.

Farley, A., Wang, J. and Marshall, J.A. (2022) "How to Pick a Mobile Robot Simulator: A Quantitative Comparison of CoppeliaSim, Gazebo, MORSE and Webots with a Focus on Accuracy of Motion," *Simulation Modelling Practice and Theory*, 120, p. 102629.

Franklin, G.F., Powell, J.D. and Emami-Naeini, A. (2015) *Feedback Control of Dynamic Systems*. 7th edn. Harlow: Pearson Education.

Gonzalez, D. *et al.* (2016) "A Review of Motion Planning Techniques for Automated Vehicles," *IEEE Transactions on Intelligent Transportation Systems*, 17(4), pp. 1135–1145.

Gonzalez, R.C. and Woods, R.E. (2018) *Digital Image Processing*. 4th edn. New York: Pearson.

Hillel, A.B. *et al.* (2014) "Recent Progress in Road and Lane Detection: A Survey," *Machine Vision and Applications*, 25(3), pp. 727–745.

Ivaldi, S. *et al.* (2015) "Tools for Simulating Humanoid Robot Dynamics: A Survey Based on User Feedback," *IEEE-RAS International Conference on Humanoid Robots*. IEEE, pp. 842–849.

Janai, J. *et al.* (2020a) "Computer Vision for Autonomous Vehicles: Problems, Datasets and State of the Art," *Foundations and Trends in Computer Graphics and Vision*, 12(1–3), pp. 1–308.

Janai, J. *et al.* (2020b) "Computer Vision for Autonomous Vehicles: Problems, Datasets and State of the Art," *Foundations and Trends in Computer Graphics and Vision*, 12(1–3), pp. 1–308.

Juliani, A. *et al.* (2018) *Unity: A General Platform for Intelligent Agents*. arXiv. Available at: https://arxiv.org/abs/1809.02627.

Khatib, O. (1986) "Real-Time Obstacle Avoidance for Manipulators and Mobile Robots," *The International Journal of Robotics Research*, 5(1), pp. 90–98.

Klancar, G. *et al.* (2017) *Wheeled Mobile Robotics: From Fundamentals Towards Autonomous Systems*. Oxford: Butterworth-Heinemann.

Lumelsky, V.J. and Stepanov, A.A. (1987) "Path-Planning Strategies for a Point Mobile Automaton Moving Amidst Unknown Obstacles of Arbitrary Shape," *Algorithmica*, 2(1), pp. 403–430.

Moravec, H. and Elfes, A. (1985) “High Resolution Maps from Wide Angle Sonar,” *Proceedings of the IEEE International Conference on Robotics and Automation*. IEEE, pp. 116–121.

Neven, D. *et al.* (2018) “Towards End-to-End Lane Detection: An Instance Segmentation Approach,” *IEEE Intelligent Vehicles Symposium*. IEEE, pp. 286–291.

Nise, N.S. (2020) *Control Systems Engineering*. 8th edn. Hoboken, NJ: John Wiley and Sons.

Ogata, K. (2010) *Modern Control Engineering*. 5th edn. Upper Saddle River, NJ: Prentice Hall.

Pakdaman, M. and Sanaatiyan, M.M. (2009) “Design and Implementation of Line Follower Robot,” *Proceedings of the Second International Conference on Computer and Electrical Engineering*. IEEE, pp. 585–590.

Rajamani, R. (2012) *Vehicle Dynamics and Control*. 2nd edn. New York: Springer.

Siciliano, B. and Khatib, O. (2016) *Springer Handbook of Robotics*. 2nd edn. Berlin: Springer.

Siegwart, R., Nourbakhsh, I.R. and Scaramuzza, D. (2011) *Introduction to Autonomous Mobile Robots*. 2nd edn. Cambridge, MA: MIT Press.

Snider, J.M. (2009) *Automatic Steering Methods for Autonomous Automobile Path Tracking*. Technical Report CMU-RI-TR-09-08. Pittsburgh, PA: Carnegie Mellon University Robotics Institute.

Thrun, S., Burgard, W. and Fox, D. (2006a) *Probabilistic Robotics*. Cambridge, MA: MIT Press.

Thrun, S., Burgard, W. and Fox, D. (2006b) *Probabilistic Robotics*. Cambridge, MA: MIT Press.

Ullrich, G. (2015a) *Automated Guided Vehicle Systems: A Primer with Practical Applications*. 2nd edn. Berlin: Springer.

Ullrich, G. (2015b) *Automated Guided Vehicle Systems: A Primer with Practical Applications*. 2nd ed. Berlin: Springer.

Visioli, A. (2006) *Practical PID Control*. London: Springer.

Winner, H. *et al.* (2016) *Handbook of Driver Assistance Systems: Basic Information, Components and Systems for Active Safety and Comfort*. Cham: Springer.

Ziegler, J.G. and Nichols, N.B. (1942) “Optimum Settings for Automatic Controllers,” *Transactions of the ASME*, 64(11), pp. 759–768.

BIBLIOGRAPHY

Åström, K.J. and Hägglund, T. (2006) *Advanced PID Control*. Research Triangle Park, NC: ISA - The Instrumentation, Systems, and Automation Society.

Åström, K.J. and Murray, R.M. (2021) *Feedback Systems: An Introduction for Scientists and Engineers*. 2nd edn. Princeton, NJ: Princeton University Press.

Borenstein, J. and Koren, Y. (1991) "The Vector Field Histogram – Fast Obstacle Avoidance for Mobile Robots," *IEEE Transactions on Robotics and Automation*, 7(3), pp. 278–288.

Canny, J. (1986) "A Computational Approach to Edge Detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 8(6), pp. 679–698.

Corke, P. (2017) *Robotics, Vision and Control: Fundamental Algorithms in MATLAB*. 2nd edn. Berlin: Springer.

Coulter, R.C. (1992) *Implementation of the Pure Pursuit Path Tracking Algorithm*. Technical Report CMU-RI-TR-92-01. Pittsburgh, PA: Carnegie Mellon University Robotics Institute.

Dudek, G. and Jenkin, M. (2010) *Computational Principles of Mobile Robotics*. 2nd ed. Cambridge: Cambridge University Press.

Farley, A., Wang, J. and Marshall, J.A. (2022) "How to Pick a Mobile Robot Simulator: A Quantitative Comparison of CoppeliaSim, Gazebo, MORSE and Webots with a Focus on Accuracy of Motion," *Simulation Modelling Practice and Theory*, 120, p. 102629.

Franklin, G.F., Powell, J.D. and Emami-Naeini, A. (2015) *Feedback Control of Dynamic Systems*. 7th edn. Harlow: Pearson Education.

Gonzalez, D. *et al.* (2016) "A Review of Motion Planning Techniques for Automated Vehicles," *IEEE Transactions on Intelligent Transportation Systems*, 17(4), pp. 1135–1145.

Gonzalez, R.C. and Woods, R.E. (2018) *Digital Image Processing*. 4th edn. New York: Pearson.

Hillel, A.B. *et al.* (2014) "Recent Progress in Road and Lane Detection: A Survey," *Machine Vision and Applications*, 25(3), pp. 727–745.

Ivaldi, S. *et al.* (2015) "Tools for Simulating Humanoid Robot Dynamics: A Survey Based on User Feedback," *IEEE-RAS International Conference on Humanoid Robots*. IEEE, pp. 842–849.

Janai, J. *et al.* (2020a) "Computer Vision for Autonomous Vehicles: Problems, Datasets and State of the Art," *Foundations and Trends in Computer Graphics and Vision*, 12(1–3), pp. 1–308.

Janai, J. *et al.* (2020b) "Computer Vision for Autonomous Vehicles: Problems, Datasets and State of the Art," *Foundations and Trends in Computer Graphics and Vision*, 12(1–3), pp. 1–308.

Juliani, A. *et al.* (2018) *Unity: A General Platform for Intelligent Agents*. arXiv. Available at: https://arxiv.org/abs/1809.02627.

Khatib, O. (1986) "Real-Time Obstacle Avoidance for Manipulators and Mobile Robots," *The International Journal of Robotics Research*, 5(1), pp. 90–98.

Klancar, G. *et al.* (2017) *Wheeled Mobile Robotics: From Fundamentals Towards Autonomous Systems*. Oxford: Butterworth-Heinemann.

Lumelsky, V.J. and Stepanov, A.A. (1987) "Path-Planning Strategies for a Point Mobile Automaton Moving Amidst Unknown Obstacles of Arbitrary Shape," *Algorithmica*, 2(1), pp. 403–430.

Moravec, H. and Elfes, A. (1985) “High Resolution Maps from Wide Angle Sonar,” *Proceedings of the IEEE International Conference on Robotics and Automation*. IEEE, pp. 116–121.

Neven, D. *et al.* (2018) “Towards End-to-End Lane Detection: An Instance Segmentation Approach,” *IEEE Intelligent Vehicles Symposium*. IEEE, pp. 286–291.

Nise, N.S. (2020) *Control Systems Engineering*. 8th edn. Hoboken, NJ: John Wiley and Sons.

Ogata, K. (2010) *Modern Control Engineering*. 5th edn. Upper Saddle River, NJ: Prentice Hall.

Pakdaman, M. and Sanaatiyan, M.M. (2009) “Design and Implementation of Line Follower Robot,” *Proceedings of the Second International Conference on Computer and Electrical Engineering*. IEEE, pp. 585–590.

Rajamani, R. (2012) *Vehicle Dynamics and Control*. 2nd edn. New York: Springer.

Siciliano, B. and Khatib, O. (2016) *Springer Handbook of Robotics*. 2nd edn. Berlin: Springer.

Siegwart, R., Nourbakhsh, I.R. and Scaramuzza, D. (2011) *Introduction to Autonomous Mobile Robots*. 2nd edn. Cambridge, MA: MIT Press.

Snider, J.M. (2009) *Automatic Steering Methods for Autonomous Automobile Path Tracking*. Technical Report CMU-RI-TR-09-08. Pittsburgh, PA: Carnegie Mellon University Robotics Institute.

Thrun, S., Burgard, W. and Fox, D. (2006a) *Probabilistic Robotics*. Cambridge, MA: MIT Press.

Thrun, S., Burgard, W. and Fox, D. (2006b) *Probabilistic Robotics*. Cambridge, MA: MIT Press.

Ullrich, G. (2015a) *Automated Guided Vehicle Systems: A Primer with Practical Applications*. 2nd edn. Berlin: Springer.

Ullrich, G. (2015b) *Automated Guided Vehicle Systems: A Primer with Practical Applications*. 2nd ed. Berlin: Springer.

Visioli, A. (2006) *Practical PID Control*. London: Springer.

Winner, H. *et al.* (2016) *Handbook of Driver Assistance Systems: Basic Information, Components and Systems for Active Safety and Comfort*. Cham: Springer.

Ziegler, J.G. and Nichols, N.B. (1942) “Optimum Settings for Automatic Controllers,” *Transactions of the ASME*, 64(11), pp. 759–768.

APPENDIX A

6.6 Main Code (Robot Controller)

```
using System;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.Serialization;

public class RobotController : MonoBehaviour
{
    [Header("Start")]
    public Vector3 startingPosition = new Vector3(0f, 0.5f, 0f);
    public float startingRotation = 0f;
    [FormerlySerializedAs("carVisualObject")]
    public Transform visualModel;
    [FormerlySerializedAs("carVisualOffset")]
    public Vector3 visualOffset = new Vector3(0f, 0.5f, 0f);

    [Header("Movement")]
    public float baseForwardSpeed = 600f;
    public float maxVelocity = 8f;
    public float maxCurveVelocity = 4f;
    public float minimumSpeed = 1.0f;
    [Range(0.5f, 3f)]
    public float brakingAggressiveness = 1.5f;

    [Header("Steering Controller")]
    public float Kp = 1.2f;
    public float Kd = 0.35f;
    public bool usePDControl = true;
    public float Kh = 1.4f;
    public float maxTurnRate = 2.5f;

    [Header("Integral Control")]
    [Tooltip("Integral gain.")]
    public float Ki = 0.08f;

    [Tooltip("Clamp for the integral accumulator.")]
    public float integralClamp = 0.45f;
```

```
[Tooltip("Integral leak per second.")]
[Range(0f, 2f)]
public float integralLeakPerSecond = 0.35f;

[Tooltip("Only integrate when the lane centre is available.")]
public bool integrateOnlyWhenHasCenter = true;

[Tooltip("Only integrate when both lane edges are visible.")]
public bool integrateOnlyWhenBothEdges = true;

[Tooltip("Disable the integral term in curves.")]
public bool disableIntegralInCurves = true;

[Tooltip("Disable the integral term during avoidance.")]
public bool disableIntegralDuringAvoidance = true;

[Tooltip("Freeze integral accumulation when steering is saturated.")]
public bool freezeIntegralWhenSaturated = true;

[Tooltip("Dead-zone scale used by the integral term.")]
[Range(0f, 1f)]
public float integralDeadZoneFactor = 0.5f;

[Tooltip("Smoothing time for the integral output.")]
[Range(0.0f, 0.5f)]
public float integralSmoothingTime = 0.08f;

[Header("Stability")]
[Range(0f, 0.15f)]
public float deadZone = 0.03f;
[Range(0f, 0.99f)]
public float derivativeSmoothing = 0.85f;
[Range(0.01f, 0.5f)]
public float errorSmoothingTime = 0.12f;
[Range(0.01f, 0.5f)]
public float headingSmoothingTime = 0.10f;

[Header("Lookahead")]
public float lookaheadMin = 1.5f;
```

```
    public float lookaheadMax = 5.0f;
    [Range(0.5f, 2f)]
    public float lookaheadSpeedFactor = 1.2f;
    public float predictionDistance = 6.0f;
    public float maxHeadingRadians = 0.8f;
    [Range(0.01f, 0.3f)]
    public float aimPointSmoothingTime = 0.08f;

    [Header("Feedforward")]
    public float Kff = 0.7f;
    [Range(0.01f, 0.3f)]
    public float feedforwardSmoothingTime = 0.10f;

    [Header("Rate Limits")]
    public float maxTurnRateChangePerSecond = 8.0f;
    public float maxAngularAcceleration = 10.0f;

    [Header("Obstacle Avoidance")]
    public LayerMask obstacleLayer;
    public float obstacleDetectDistance = 16.0f;
    public float obstacleAvoidStartDistance = 10.0f;
    public float obstacleStopDistance = 2.5f;
    public float obstacleSphereRadius = 0.35f;
    public float obstacleSensorForwardOffset = 1.4f;
    public float obstacleSensorHeight = 0.75f;

    [Header("Obstacle Fan Sensors")]
    public int obstacleFanRayCount = 9;
    [Range(5f, 60f)]
    public float obstacleFanHalfAngle = 25f;
    [Tooltip("Lateral spacing between the three sensor columns.")]
    public float obstacleFanLateralSpread = 0.5f;

    [Header("Avoidance Stability")]
    [Tooltip("Scale Kp during avoidance.")]
    [Range(0.1f, 1f)]
    public float avoidKpScale = 0.5f;

    [Tooltip("Scale Kh during avoidance.")]
    [Range(0.1f, 1f)]
```

```
public float avoidKhScale = 0.6f;

[Tooltip("Boost Kd during avoidance.")]
[Range(1f, 3f)]
public float avoidKdBoost = 1.8f;

[Tooltip("Turn-rate limit during avoidance.")]
[Range(0.5f, 3f)]
public float avoidMaxTurnRate = 1.8f;

[Tooltip("Rate-of-change limit during avoidance.")]
public float avoidTurnRateChangePerSecond = 5.0f;
public float maxDodgeLeftOffset = 1.0f;
public float laneEdgeSafetyMargin = 0.45f;
public float dodgeSmoothTime = 0.50f;
public float mergeSmoothTime = 0.65f;
public float obstacleDodgeSpeed = 3.0f;
public float obstacleMaxBrakeAccel = 18f;
public float obstacleBrakeGain = 8f;
public float obstacleReactionTime = 0.25f;
public bool checkLeftPath = true;
public float leftPathCheckExtra = 1.0f;

[Header("Side Sensor")]
public float sideSensorRange = 3.0f;
public float sideSensorRadius = 0.3f;
public float sideSensorHeight = 0.6f;
public int sideSensorCount = 3;
public float sideSensorLengthSpread = 1.5f;
public float sideClearConfirmTime = 0.3f;

[Header("Lane Sensors")]
public float sensorForwardDistance = 1.2f;
public float sensorHeightOffset = 0.5f;
public float raycastDownDistance = 3.0f;
public float expectedLaneWidth = 4.0f;

[Header("Lane Sensor Array")]
public int sensorsPerSide = 5;
public float sensorSpread = 2.5f;
```

```
[Header("Lane Layers")]
public LayerMask leftLaneLayer;
public LayerMask rightLaneLayer;

[Header("Lane Robustness")]
[Range(0f, 1f)]
public float widthBlendToMeasured = 0.6f;

[Header("Curve Detection")]
[Range(0f, 1f)]
public float curveEnterThreshold = 0.10f;
[Range(0f, 1f)]
public float curveExitThreshold = 0.06f;
[Range(0.1f, 1f)]
public float curveExitHoldTime = 0.4f;

[Header("Recovery")]
public float recoveryTimeout = 5.0f;
public float recoverySteeringMultiplier = 1.0f;
public bool keepMovingDuringRecovery = true;

[Header("Debug")]
public bool showDebugRays = true;
public bool showLookaheadRays = true;
public bool showPredictionRays = false;
public bool showObstacleRays = true;
public bool showDebugLog = false;

private Rigidbody rb;

private float currentLateralError = 0f;
private float previousLateralError = 0f;
private float smoothedError = 0f;
private float smoothedDerivative = 0f;

private float currentHeadingError = 0f;
private float smoothedHeadingError = 0f;

private float appliedTurnRate = 0f;
```

```
private float rateLimitedTurnRate = 0f;
private float dynamicLookahead = 2.5f;

private Vector3 smoothedAimLocal = new Vector3(0f, 0f, 2f);
private float smoothedYawRateFF = 0f;

private float targetVelocity;
private float smoothedTargetVelocity;

private bool leftLineDetected = false;
private bool rightLineDetected = false;
private float leftLineDistance = 0f;
private float rightLineDistance = 0f;

private float lastKnownError = 0f;
private float timeSinceLineDetected = 0f;
private bool isRecovering = false;

private bool isInCurve = false;
private float curveExitCounter = 0f;
private float predictedCurvature = 0f;

private float avoidOffsetX = 0f;
private float avoidOffsetVelocity = 0f;
private float obstacleSpeedCap = float.PositiveInfinity;
private bool obstacleMustStop = false;
private float lastObstacleDistance = 999f;

private enum AvoidState { None, Dodging, Passing, Merging }
private AvoidState avoidState = AvoidState.None;
private AvoidState previousAvoidState = AvoidState.None;
private float sideClearTimer = 0f;
private bool suppressRightLaneSensors = false;

private LaneSample nearSample;
private LaneSample farSample;
private LaneSample predictionSample;

private float integralError = 0f;
private float integralErrorSmoothed = 0f;
```

```
    private float integralVelocity = 0f;

    [HideInInspector] public float debugPTerm;
    [HideInInspector] public float debugITerm;
    [HideInInspector] public float debugDTerm;
    [HideInInspector] public float debugHTerm;
    [HideInInspector] public float debugFFTerm;

    void Start()
    {
        rb = GetComponent<Rigidbody>();
        if (rb == null)
        {
            Debug.LogError("[RobotController] Rigidbody missing!");
            return;
        }

        // Keep the robot upright so the tests focus on lane-following behaviour rather than body roll.
        rb.constraints = RigidbodyConstraints.FreezeRotationX | RigidbodyConstraints.FreezeRotationZ;
        rb.interpolation = RigidbodyInterpolation.Interpolate;

        transform.position = startingPosition;
        transform.rotation = Quaternion.Euler(0f, startingRotation, 0f);
        rb.linearVelocity = Vector3.zero;
        rb.angularVelocity = Vector3.zero;

        smoothedAimLocal = new Vector3(0f, 0f, lookaheadMin);
        targetVelocity = maxVelocity;
        smoothedTargetVelocity = maxVelocity;

        integralError = 0f;
        integralErrorSmoothed = 0f;
        integralVelocity = 0f;

        Debug.Log("[RobotController v10] Started - PID lateral (I-term) + anti-windup + gated integral");
    }

    void FixedUpdate()
    {
        if (rb == null) return;
```

```
        UpdateVisualModel();

        float speed = rb.linearVelocity.magnitude;
        float speed01 = Mathf.Clamp01(speed / Mathf.Max(0.01f, maxVelocity));

        // Increase lookahead with speed, then shorten it again in curves so the robot reacts earlier.
        float baseLookahead = Mathf.Lerp(lookaheadMin, lookaheadMax, speed01 * lookaheadSpeedFactor);
        if (isInCurve) baseLookahead *= 0.7f;
        dynamicLookahead = Mathf.Max(lookaheadMin, baseLookahead);

        // Three lane samples are used: near for centring, far for heading, and prediction for future curvature.
        nearSample = SampleLane(sensorForwardDistance, showDebugRays, 0);
        farSample = SampleLane(dynamicLookahead, showLookaheadRays, 1);
        predictionSample = SampleLane(predictionDistance, showPredictionRays, 2);

        UpdateDetectionFlags();

        previousAvoidState = avoidState;
        UpdateObstacleAvoidance(speed);

        PredictUpcomingCurvature();
        CalculateErrors();
        CalculateTargetSpeed();
        ApplyControl();
    }

    private void UpdateVisualModel()
    {
        if (visualModel != null)
            visualModel.localPosition = visualOffset;
    }

    // Each sample stores what the lane sensors saw at one distance ahead of the robot.
    private struct LaneSample
    {
        public bool leftDetected;
        public bool rightDetected;
```

```
        public bool hasCenter;
        public float leftOffset;
        public float rightOffset;
        public float halfWidthUsed;
        public Vector3 centerWorld;
        public Vector3 centerLocal;
    }

    private LaneSample SampleLane(float forwardDist, bool drawRays, int rayTint)
    {
        LaneSample sample = new LaneSample
        {
            leftDetected = false,
            rightDetected = false,
            hasCenter = false,
            leftOffset = 0f,
            rightOffset = 0f,
            halfWidthUsed = Mathf.Max(0.001f, expectedLaneWidth * 0.5f),
            centerWorld = transform.position + transform.forward * forwardDist,
            centerLocal = new Vector3(0f, 0f, forwardDist)
        };

        Vector3 sensorOrigin = transform.position + transform.forward * forwardDist;
        sensorOrigin.y = transform.position.y + sensorHeightOffset;

        Color[] tintColors = { Color.green, Color.cyan, Color.magenta };
        Color[] missColorsL = { Color.white, new Color(1f, 1f, 1f, 0.4f), new Color(1f, 0.5f, 1f, 0.3f) };
        Color[] missColorsR = { Color.yellow, new Color(1f, 1f, 0f, 0.4f), new Color(1f, 1f, 0.5f, 0.3f) };

        // Multiple rays per side make the estimate more robust when one or two sensors miss the lane line.
        List<float> leftOffsets = new List<float>(sensorsPerSide);
        List<Vector3> leftPoints = new List<Vector3>(sensorsPerSide);
        List<float> rightOffsets = new List<float>(sensorsPerSide);
        List<Vector3> rightPoints = new List<Vector3>(sensorsPerSide);

        for (int i = 0; i < sensorsPerSide; i++)
        {
            float t = (sensorsPerSide > 1) ? (float)i / (sensorsPerSide - 1) : 0f;
            float offset = Mathf.Lerp(0.3f, sensorSpread, t);
```

```
        Vector3 leftPos = sensorOrigin - transform.right * offset;
        if (Physics.Raycast(leftPos, Vector3.down, out RaycastHit leftHit, raycastDownDistance, leftLaneLayer, QueryTriggerInteraction.Ignore))
        {
            sample.leftDetected = true;
            leftOffsets.Add(offset);
            leftPoints.Add(leftHit.point);
            if (drawRays) Debug.DrawLine(leftPos, leftHit.point, tintColors[rayTint]);
        }
        else if (drawRays)
        {
            Debug.DrawRay(leftPos, Vector3.down * raycastDownDistance, missColorsL[rayTint]);
        }

        if (!suppressRightLaneSensors)
        {
            Vector3 rightPos = sensorOrigin + transform.right * offset;
            if (Physics.Raycast(rightPos, Vector3.down, out RaycastHit rightHit, raycastDownDistance, rightLaneLayer, QueryTriggerInteraction.Ignore))
            {
                sample.rightDetected = true;
                rightOffsets.Add(offset);
                rightPoints.Add(rightHit.point);
                if (drawRays) Debug.DrawLine(rightPos, rightHit.point, tintColors[rayTint]);
            }
            else if (drawRays)
            {
                Debug.DrawRay(rightPos, Vector3.down * raycastDownDistance, missColorsR[rayTint]);
            }
        }
    }

    Vector3 leftPoint = Vector3.zero;
    Vector3 rightPoint = Vector3.zero;
    bool hasLeft = sample.leftDetected && leftOffsets.Count > 0;
    bool hasRight = sample.rightDetected && rightOffsets.Count > 0;

    if (hasLeft)
    {
```

```
            // Median selection rejects single outliers better than taking the widest or average hit directly.
            int idx = GetMedianIndex(leftOffsets);
            sample.leftOffset = leftOffsets[idx];
            leftPoint = leftPoints[idx];
        }

        if (hasRight)
        {
            int idx = GetMedianIndex(rightOffsets);
            sample.rightOffset = rightOffsets[idx];
            rightPoint = rightPoints[idx];
        }

        // If only one edge is visible, rebuild the lane centre from the expected width instead of dropping it.
        if (hasLeft && hasRight)
        {
            sample.centerWorld = (leftPoint + rightPoint) * 0.5f;
            float measuredHalf = (sample.leftOffset + sample.rightOffset) * 0.5f;
            float expectedHalf = expectedLaneWidth * 0.5f;
            sample.halfWidthUsed = Mathf.Lerp(expectedHalf, measuredHalf, widthBlendToMeasured);
            sample.hasCenter = true;
        }
        else if (hasLeft)
        {
            sample.centerWorld = leftPoint + transform.right * (expectedLaneWidth * 0.5f);
            sample.halfWidthUsed = expectedLaneWidth * 0.5f;
            sample.hasCenter = true;
        }
        else if (hasRight)
        {
            sample.centerWorld = rightPoint - transform.right * (expectedLaneWidth * 0.5f);
            sample.halfWidthUsed = expectedLaneWidth * 0.5f;
            sample.hasCenter = true;
        }

        if (sample.hasCenter)
            sample.centerLocal = transform.InverseTransformPoint(sample.centerWorld);

        return sample;
    }
```

```
    private int GetMedianIndex(List<float> values)
    {
        if (values.Count <= 1) return 0;

        List<int> indices = new List<int>(values.Count);
        for (int i = 0; i < values.Count; i++) indices.Add(i);
        indices.Sort((a, b) => values[a].CompareTo(values[b]));
        return indices[indices.Count / 2];
    }

    private void UpdateDetectionFlags()
    {
        leftLineDetected = nearSample.leftDetected;
        rightLineDetected = nearSample.rightDetected;
        leftLineDistance = nearSample.leftOffset;
        rightLineDistance = nearSample.rightOffset;
    }

    private void UpdateObstacleAvoidance(float speed)
    {
        float dt = Time.fixedDeltaTime;
        obstacleMustStop = false;
        obstacleSpeedCap = float.PositiveInfinity;

        Vector3 originBase = transform.position + Vector3.up * obstacleSensorHeight + transform.forward
* obstacleSensorForwardOffset;

        float brakeAccel = Mathf.Max(0.1f, obstacleMaxBrakeAccel);
        float reactionDist = speed * Mathf.Max(0.0f, obstacleReactionTime);
        float stopDist = (speed * speed) / (2f * brakeAccel);
        float shiftDist = speed * Mathf.Max(0.10f, dodgeSmoothTime * 2.0f);
        // Start avoidance earlier at higher speed so there is enough distance for braking and lateral
motion.
        float dynamicStartDist = Mathf.Max(obstacleAvoidStartDistance, reactionDist + stopDist + 0.7f,
shiftDist + 0.8f);
        float detectDist = Mathf.Max(dynamicStartDist + 2f, obstacleDetectDistance);

        bool forwardHit = false;
        float forwardDist = float.PositiveInfinity;
```

```
int raysPerColumn = Mathf.Max(1, obstacleFanRayCount / 3);
float[] lateralOffsets = { -obstacleFanLateralSpread, 0f, +obstacleFanLateralSpread };

for (int col = 0; col < 3; col++)
{
   Vector3 colOrigin = originBase + transform.right * lateralOffsets[col];

   for (int r = 0; r < raysPerColumn; r++)
   {
      float t = (raysPerColumn > 1) ? (float)r / (raysPerColumn - 1) : 0.5f;
      float angle = Mathf.Lerp(-obstacleFanHalfAngle, +obstacleFanHalfAngle, t);
      Vector3 rayDir = Quaternion.AngleAxis(angle, Vector3.up) * transform.forward;

      bool hit = Physics.SphereCast(colOrigin, obstacleSphereRadius, rayDir, out RaycastHit hitInfo, detectDist, obstacleLayer, QueryTriggerInteraction.Ignore);

      if (showObstacleRays)
      {
         Color rayColor = hit ? Color.red : new Color(1f, 0.2f, 0.2f, 0.15f);
         Debug.DrawRay(colOrigin, rayDir * detectDist, rayColor);
         if (hit) Debug.DrawLine(colOrigin, hitInfo.point, Color.red);
      }

      if (hit)
      {
         // Use forward distance rather than diagonal ray distance.
         float angleRad = angle * Mathf.Deg2Rad;
         float projectedDist = hitInfo.distance * Mathf.Cos(angleRad);

         if (projectedDist < forwardDist)
         {
            forwardHit = true;
            forwardDist = projectedDist;
         }
      }
   }
}

if (forwardHit)
```

```
        lastObstacleDistance = forwardDist;

    bool sideSeesObstacle = false;
    if (avoidState == AvoidState.Dodging || avoidState == AvoidState.Passing)
        sideSeesObstacle = HasSideObstacle();

    float targetOffset = 0f;

    // Main avoidance sequence: start dodge, hold the offset while passing, then merge back when clear.
    switch (avoidState)
    {
        case AvoidState.None:
            suppressRightLaneSensors = false;
            if (forwardHit && forwardDist <= dynamicStartDist)
            {
                avoidState = AvoidState.Dodging;
                suppressRightLaneSensors = true;
                sideClearTimer = 0f;
            }
            break;

        case AvoidState.Dodging:
            // Ignore the right lane during the dodge so the controller does not merge back too early.
            suppressRightLaneSensors = true;

            if (forwardHit)
            {
                float d = forwardDist;

                if (d <= obstacleStopDistance)
                {
                    if (checkLeftPath)
                    {
                        // Final left-path check prevents committing to the dodge if the escape path is blocked.
                        Vector3 leftPathOrigin = originBase + transform.right * (-maxDodgeLeftOffset);
                        float leftCheckDist = Mathf.Min(detectDist, d + leftPathCheckExtra);
                        bool leftBlocked = Physics.SphereCast(leftPathOrigin, obstacleSphereRadius, transform.forward, out RaycastHit leftHit, leftCheckDist, obstacleLayer, QueryTriggerInteraction.Ignore);
```

```
                if (showObstacleRays)
                {
                    Color lc = leftBlocked ? new Color(1f, 0.5f, 0f, 1f) : new Color(0.2f, 1f, 0.4f, 0.8f);
                    Debug.DrawRay(leftPathOrigin, transform.forward * leftCheckDist, lc);
                }

                if (leftBlocked)
                {
                    obstacleMustStop = true;
                    obstacleSpeedCap = 0f;
                    break;
                }
            }
        }

        float t = Mathf.InverseLerp(dynamicStartDist, obstacleStopDistance, d);
        t = Mathf.Clamp01(t);
        targetOffset = -Mathf.Lerp(0f, maxDodgeLeftOffset, t);

        float halfWidth = nearSample.hasCenter ? nearSample.halfWidthUsed :
(expectedLaneWidth * 0.5f);
        float maxOffsetInsideLane = Mathf.Max(0.1f, halfWidth - laneEdgeSafetyMargin);
        targetOffset = Mathf.Clamp(targetOffset, -maxOffsetInsideLane, +maxOffsetInsideLane);

        if (!obstacleMustStop)
        {
            float distToStopMargin = Mathf.Max(0f, d - obstacleStopDistance);
            float vAllow = Mathf.Sqrt(2f * brakeAccel * distToStopMargin);
            float capWanted = Mathf.Lerp(maxVelocity, obstacleDodgeSpeed, t * t);
            obstacleSpeedCap = Mathf.Min(capWanted, vAllow);
        }
    }
    else
    {
        targetOffset = -maxDodgeLeftOffset;
        float halfWidth = nearSample.hasCenter ? nearSample.halfWidthUsed :
(expectedLaneWidth * 0.5f);
        float maxOffsetInsideLane = Mathf.Max(0.1f, halfWidth - laneEdgeSafetyMargin);
        targetOffset = Mathf.Clamp(targetOffset, -maxOffsetInsideLane, +maxOffsetInsideLane);
```

```
            obstacleSpeedCap = obstacleDodgeSpeed;
        }

        if (sideSeesObstacle)
        {
            avoidState = AvoidState.Passing;
            sideClearTimer = 0f;
        }
        else if (!forwardHit)
        {
            sideClearTimer += dt;
            if (sideClearTimer > 0.5f)
                avoidState = AvoidState.Merging;
        }
        else
        {
            sideClearTimer = 0f;
        }
        break;

    case AvoidState.Passing:
        suppressRightLaneSensors = true;

        targetOffset = -maxDodgeLeftOffset;
        float passHalfWidth = nearSample.hasCenter ? nearSample.halfWidthUsed :
(expectedLaneWidth * 0.5f);
        float passMaxOffsetInsideLane = Mathf.Max(0.1f, passHalfWidth - laneEdgeSafetyMargin);
        targetOffset = Mathf.Clamp(targetOffset, -passMaxOffsetInsideLane,
+passMaxOffsetInsideLane);
        obstacleSpeedCap = obstacleDodgeSpeed;

        if (!sideSeesObstacle)
        {
            sideClearTimer += dt;
            if (sideClearTimer >= sideClearConfirmTime)
                avoidState = AvoidState.Merging;
        }
        else
        {
            sideClearTimer = 0f;
```

```
            }

            if (forwardHit && forwardDist < obstacleStopDistance)
            {
                obstacleMustStop = true;
                obstacleSpeedCap = 0f;
            }
            break;

        case AvoidState.Merging:
            targetOffset = 0f;

            if (Mathf.Abs(avoidOffsetX) < 0.15f)
            {
                suppressRightLaneSensors = false;
                avoidState = AvoidState.None;
            }
            else
            {
                suppressRightLaneSensors = true;
            }

            if (forwardHit && forwardDist <= dynamicStartDist)
            {
                avoidState = AvoidState.Dodging;
                suppressRightLaneSensors = true;
                sideClearTimer = 0f;
            }
            break;
    }

    float currentSmoothTime = (avoidState == AvoidState.Merging) ? mergeSmoothTime : dodgeSmoothTime;
    avoidOffsetX = Mathf.SmoothDamp(avoidOffsetX, targetOffset, ref avoidOffsetVelocity, currentSmoothTime);
  }

  private bool HasSideObstacle()
  {
    bool hitAny = false;
```

```
        // Side checks confirm when the obstacle is alongside the robot rather than only in front of it.
        for (int i = 0; i < sideSensorCount; i++)
        {
            float t = (sideSensorCount > 1) ? (float)i / (sideSensorCount - 1) : 0.5f;
            float forwardOffset = Mathf.Lerp(-sideSensorLengthSpread, sideSensorLengthSpread, t);

            Vector3 origin = transform.position + Vector3.up * sideSensorHeight + transform.forward * forwardOffset;
            Vector3 direction = transform.right;

            bool hit = Physics.SphereCast(origin, sideSensorRadius, direction, out RaycastHit hitInfo, sideSensorRange, obstacleLayer, QueryTriggerInteraction.Ignore);

            if (showObstacleRays)
            {
                Color c = hit ? new Color(1f, 0.6f, 0f, 1f) : new Color(0.3f, 0.8f, 1f, 0.5f);
                Debug.DrawRay(origin, direction * sideSensorRange, c);
                if (hit) Debug.DrawLine(origin, hitInfo.point, Color.yellow);
            }

            if (hit) hitAny = true;
        }

        return hitAny;
    }

    private void PredictUpcomingCurvature()
    {
        if (nearSample.hasCenter && farSample.hasCenter)
        {
            Vector3 nearLocal = nearSample.centerLocal;
            Vector3 farLocal = farSample.centerLocal;

            // Compare how the lane centre moves between near and far samples to estimate curvature.
            float lateralDeviation = farLocal.x - nearLocal.x;
            float longitudinalDist = Mathf.Max(0.1f, farLocal.z - nearLocal.z);

            float curvatureEstimate = lateralDeviation / (longitudinalDist * longitudinalDist);
```

```
        if (predictionSample.hasCenter)
        {
            // A third sample further ahead helps stabilise the sign and strength on S-curves.
            Vector3 predLocal = predictionSample.centerLocal;
            float predLateral = predLocal.x - farLocal.x;
            float predLongitudinal = Mathf.Max(0.1f, predLocal.z - farLocal.z);
            float predCurvature = predLateral / (predLongitudinal * predLongitudinal);

            curvatureEstimate = Mathf.Max(Mathf.Abs(curvatureEstimate), Mathf.Abs(predCurvature)) *
Mathf.Sign(curvatureEstimate + predCurvature);
        }

        predictedCurvature = Mathf.Lerp(predictedCurvature, curvatureEstimate, ExpAlpha(0.15f));
    }

    float absError = Mathf.Abs(smoothedError);
    float absHeading = Mathf.Abs(smoothedHeadingError);
    float absCurvature = Mathf.Abs(predictedCurvature);

    if (!isInCurve)
    {
        if (absError > curveEnterThreshold ||
            absHeading > maxHeadingRadians * 0.3f ||
            absCurvature > 0.15f)
        {
            isInCurve = true;
            curveExitCounter = 0f;
        }
    }
    else
    {
        // Hysteresis prevents fast switching at the threshold.
        if (absError < curveExitThreshold &&
            absHeading < maxHeadingRadians * 0.15f &&
            absCurvature < 0.08f)
        {
            curveExitCounter += Time.fixedDeltaTime;
            if (curveExitCounter >= curveExitHoldTime)
            {
                isInCurve = false;
```

```
                curveExitCounter = 0f;
            }
        }
        else
        {
            curveExitCounter = 0f;
        }
    }
}

private void CalculateErrors()
{
    previousLateralError = currentLateralError;

    if (nearSample.hasCenter)
    {
        isRecovering = false;

        // Normalise by half-lane width so controller thresholds stay consistent across width estimates.
        float rawLateral = (nearSample.centerLocal.x + avoidOffsetX) / Mathf.Max(0.001f, nearSample.halfWidthUsed);
        currentLateralError = Mathf.Clamp(rawLateral, -1f, 1f);

        Vector3 aimWorld = farSample.hasCenter ? farSample.centerWorld : nearSample.centerWorld;
        Vector3 aimLocal = transform.InverseTransformPoint(aimWorld);
        aimLocal.x += avoidOffsetX;

        smoothedAimLocal = Vector3.Lerp(smoothedAimLocal, aimLocal, ExpAlpha(aimPointSmoothingTime));

        float rawHeading = Mathf.Atan2(smoothedAimLocal.x, Mathf.Max(0.001f, smoothedAimLocal.z));
        currentHeadingError = Mathf.Clamp(rawHeading, -maxHeadingRadians, maxHeadingRadians);

        timeSinceLineDetected = 0f;
        lastKnownError = currentLateralError;
    }
    else
    {
```

```
        // Recovery keeps steering from the last reliable estimate instead of snapping to zero immediately.
        isRecovering = true;
        timeSinceLineDetected += Time.fixedDeltaTime;

        if (timeSinceLineDetected < recoveryTimeout)
        {
            currentLateralError = Mathf.Clamp(lastKnownError * recoverySteeringMultiplier, -1f, 1f);
            currentHeadingError = 0f;
            isInCurve = true;
            curveExitCounter = 0f;
        }
        else if (!keepMovingDuringRecovery)
        {
            currentLateralError = 0f;
            currentHeadingError = 0f;
            rb.linearVelocity = Vector3.zero;
            rb.angularVelocity = Vector3.zero;
            Debug.LogError("[TIMEOUT] Robot stopped");
        }
    }

    smoothedError = Mathf.Lerp(smoothedError, currentLateralError, ExpAlpha(errorSmoothingTime));
    smoothedHeadingError = Mathf.Lerp(smoothedHeadingError, currentHeadingError, ExpAlpha(headingSmoothingTime));

    float rawDerivative = (currentLateralError - previousLateralError) / Time.fixedDeltaTime;
    rawDerivative = Mathf.Clamp(rawDerivative, -10f, 10f);
    smoothedDerivative = Mathf.Lerp(smoothedDerivative, rawDerivative, 1f - derivativeSmoothing);
}

private void CalculateTargetSpeed()
{
    float target = maxVelocity;

    float curvatureFactor = Mathf.Abs(predictedCurvature) * brakingAggressiveness;
    float headingFactor = Mathf.Abs(smoothedHeadingError) / maxHeadingRadians;
    float errorFactor = Mathf.Abs(smoothedError);
```

```
        // Slow down from the worst of curvature, heading and lateral error to preserve steering authority.
        float demand = Mathf.Clamp01(Mathf.Max(curvatureFactor, Mathf.Max(headingFactor * 0.8f, errorFactor * 0.5f)));
        float speedReduction = demand * demand;

        target = Mathf.Lerp(maxVelocity, maxCurveVelocity, speedReduction);

        if (obstacleMustStop) target = 0f;
        else if (!float.IsPositiveInfinity(obstacleSpeedCap)) target = Mathf.Min(target, obstacleSpeedCap);

        if (!obstacleMustStop)
            target = Mathf.Max(target, minimumSpeed);

        targetVelocity = target;

        float smoothTime = isInCurve ? 0.2f : 0.4f;
        smoothedTargetVelocity = Mathf.Lerp(smoothedTargetVelocity, targetVelocity, ExpAlpha(smoothTime));
    }

    private bool IntegralAllowed(float eUsed)
    {
        // The I-term is only trusted when the lane estimate is stable; otherwise it can wind up on bad data.
        if (Ki <= 0.00001f) return false;
        if (isRecovering) return false;

        if (integrateOnlyWhenHasCenter && !nearSample.hasCenter) return false;
        if (integrateOnlyWhenBothEdges && !(leftLineDetected && rightLineDetected)) return false;

        if (disableIntegralInCurves && isInCurve) return false;
        if (disableIntegralDuringAvoidance && (avoidState != AvoidState.None)) return false;

        float dz = Mathf.Max(0f, deadZone) * Mathf.Clamp01(integralDeadZoneFactor);
        if (Mathf.Abs(eUsed) < dz) return false;

        return true;
    }

    private void ResetIntegral()
    {
```

```
        integralError = 0f;
        integralErrorSmoothed = 0f;
        integralVelocity = 0f;
    }

    private void ApplyControl()
    {
        if (rb == null) return;

        float dt = Time.fixedDeltaTime;

        Vector3 vel = rb.linearVelocity;
        float speed = vel.magnitude;
        float forwardSpeed = Vector3.Dot(vel, transform.forward);

        float e = smoothedError;
        float h = smoothedHeadingError;

        if (Mathf.Abs(e) < deadZone && !isInCurve && !isRecovering)
            e = 0f;

        if (avoidState != previousAvoidState) ResetIntegral();
        if (!nearSample.hasCenter) ResetIntegral();
        if (isRecovering) ResetIntegral();

        // During avoidance the controller uses softer gains and tighter limits to keep the lane change stable.
        bool isAvoiding = (avoidState != AvoidState.None);
        float effectiveKp = isAvoiding ? (Kp * avoidKpScale) : Kp;
        float effectiveKh = isAvoiding ? (Kh * avoidKhScale) : Kh;
        float effectiveKdBoost = isAvoiding ? avoidKdBoost : 1f;
        float effectiveMaxTurn = isAvoiding ? avoidMaxTurnRate : maxTurnRate;
        float effectiveTurnRateChange = isAvoiding ? avoidTurnRateChangePerSecond : maxTurnRateChangePerSecond;

        // Final steering combines lateral correction, damping, heading alignment and pure-pursuit feedforward.
        float pTerm = effectiveKp * e;

        float dTerm = 0f;
```

```
        if (usePDControl && Mathf.Abs(e) >= deadZone)
        {
            float speedAdaptiveKd = Kd * effectiveKdBoost * (1f + 0.3f * (speed / Mathf.Max(0.01f, maxVelocity)));
            dTerm = speedAdaptiveKd * smoothedDerivative;
        }

        float hTerm = effectiveKh * h;

        float yawRateFF = 0f;
        if (Kff > 0.0001f && smoothedAimLocal.z > 0.05f)
        {
            yawRateFF = ComputePurePursuitYawRate(smoothedAimLocal, speed);
            yawRateFF = Mathf.Clamp(yawRateFF, -maxTurnRate, maxTurnRate);
        }
        smoothedYawRateFF = Mathf.Lerp(smoothedYawRateFF, yawRateFF, ExpAlpha(feedforwardSmoothingTime));
        float ffTerm = Kff * smoothedYawRateFF;

        bool allowed = IntegralAllowed(e);

        // The integral term acts more like a small straight-line bias corrector than a dominant steering term.
        if (integralLeakPerSecond > 0f)
        {
            float leak = integralLeakPerSecond * dt;
            integralError = Mathf.MoveTowards(integralError, 0f, leak);
        }

        float integralCandidate = integralError;
        if (allowed)
        {
            integralCandidate = integralError + (e * dt);
            integralCandidate = Mathf.Clamp(integralCandidate, -integralClamp, integralClamp);
        }

        if (integralSmoothingTime > 0.0001f)
        {
            integralErrorSmoothed = Mathf.SmoothDamp(integralErrorSmoothed, integralCandidate, ref integralVelocity, integralSmoothingTime);
```

```
        }
        else
        {
            integralErrorSmoothed = integralCandidate;
        }

        float iTerm = Ki * integralErrorSmoothed;

        float rawTurnRate = pTerm + iTerm + dTerm + hTerm + ffTerm;
        debugPTerm = pTerm;
        debugITerm = iTerm;
        debugDTerm = dTerm;
        debugHTerm = hTerm;
        debugFFTerm = ffTerm;
        float targetTurnRate = Mathf.Clamp(rawTurnRate, -effectiveMaxTurn, effectiveMaxTurn);

        if (freezeIntegralWhenSaturated && allowed)
        {
            // Anti-windup: freeze accumulation when the output is already clamped and still pushing the same way.
            bool saturated = Mathf.Abs(rawTurnRate) > effectiveMaxTurn * 0.995f;
            bool pushingSameWay = Mathf.Sign(rawTurnRate) == Mathf.Sign(e);

            if (saturated && pushingSameWay)
            {
                integralCandidate = integralError;

                if (integralSmoothingTime > 0.0001f)
                    integralErrorSmoothed = Mathf.SmoothDamp(integralErrorSmoothed, integralCandidate, ref integralVelocity, integralSmoothingTime);
                else
                    integralErrorSmoothed = integralCandidate;

                iTerm = Ki * integralErrorSmoothed;

                rawTurnRate = pTerm + iTerm + dTerm + hTerm + ffTerm;
                targetTurnRate = Mathf.Clamp(rawTurnRate, -effectiveMaxTurn, effectiveMaxTurn);
            }
            else
            {
```

```
            integralError = integralCandidate;
        }
    }
    else
    {
        if (allowed) integralError = integralCandidate;
    }

    // Apply rate limits in two stages so the rigidbody sees a smooth yaw command rather than a step change.
    float cmdStep = Mathf.Max(0.01f, effectiveTurnRateChange) * dt;
    rateLimitedTurnRate = Mathf.MoveTowards(rateLimitedTurnRate, targetTurnRate, cmdStep);

    float outStep = Mathf.Max(0.01f, maxAngularAcceleration) * dt;
    appliedTurnRate = Mathf.MoveTowards(appliedTurnRate, rateLimitedTurnRate, outStep);

    rb.angularVelocity = new Vector3(0f, appliedTurnRate, 0f);

    float desired = smoothedTargetVelocity;

    // Forward motion is handled with simple drive/brake forces instead of setting the velocity directly.
    if (desired <= 0.01f)
    {
        float brake = Mathf.Clamp(obstacleBrakeGain * Mathf.Max(0f, forwardSpeed), 0f, obstacleMaxBrakeAccel);
        rb.AddForce(-transform.forward * brake, ForceMode.Acceleration);

        if (speed < 0.2f)
            rb.linearVelocity = Vector3.zero;
    }
    else
    {
        float forwardForce = baseForwardSpeed;

        if (speed > desired * 1.05f) forwardForce *= 0.20f;
        else if (speed > desired * 0.95f) forwardForce *= 0.60f;

        rb.AddForce(transform.forward * forwardForce * dt, ForceMode.Acceleration);

        if (forwardSpeed > desired)
```

```
        {
            float brake = Mathf.Clamp(obstacleBrakeGain * (forwardSpeed - desired), 0f, obstacleMaxBrakeAccel);
            rb.AddForce(-transform.forward * brake, ForceMode.Acceleration);
        }
    }

    if (desired > 0.1f && speed > desired * 1.18f)
    {
        Vector3 clampedVel = rb.linearVelocity.normalized * desired;
        clampedVel.y = rb.linearVelocity.y;
        rb.linearVelocity = Vector3.Lerp(rb.linearVelocity, clampedVel, 0.14f);
    }

    if (desired > 0.1f && speed < minimumSpeed && (isRecovering || isInCurve))
    {
        Vector3 minVel = transform.forward * minimumSpeed;
        minVel.y = rb.linearVelocity.y;
        rb.linearVelocity = Vector3.Lerp(rb.linearVelocity, minVel, 0.15f);
    }

    if (showDebugLog)
    {
        Debug.Log($"[v10 PID] E:{e:F3} Iacc:{integralErrorSmoothed:F3} P:{pTerm:F3} I:{iTerm:F3} D:{dTerm:F3} " +
                  $"H:{hTerm:F3} FF:{ffTerm:F3} Turn:{appliedTurnRate:F3} V:{speed:F2} Target:{desired:F2} Avoid:{avoidState}");
    }
}

private float ComputePurePursuitYawRate(Vector3 aimLocal, float speed)
{
    // Pure pursuit converts the lookahead point into a curvature request in the robot frame.
    float x = aimLocal.x;
    float z = aimLocal.z;
    float L2 = x * x + z * z;
    if (L2 < 0.0001f) return 0f;

    float kappa = 2f * x / L2;
    return speed * kappa;
```

```
    }

    private float ExpAlpha(float timeConstant)
    {
        // Convert a time constant into a frame-rate-independent interpolation factor.
        float tau = Mathf.Max(0.0001f, timeConstant);
        return 1f - Mathf.Exp(-Time.fixedDeltaTime / tau);
    }

    private void OnDrawGizmosSelected()
    {
        Vector3 originNear = transform.position + transform.forward * sensorForwardDistance;
        originNear.y = transform.position.y + sensorHeightOffset;

        for (int i = 0; i < sensorsPerSide; i++)
        {
            float t = (sensorsPerSide > 1) ? (float)i / (sensorsPerSide - 1) : 0f;
            float offset = Mathf.Lerp(0.3f, sensorSpread, t);

            Gizmos.color = Color.white;
            Gizmos.DrawRay(originNear - transform.right * offset, Vector3.down * raycastDownDistance);

            Gizmos.color = Color.yellow;
            Gizmos.DrawRay(originNear + transform.right * offset, Vector3.down * raycastDownDistance);
        }

        Gizmos.color = Color.cyan;
        Gizmos.DrawWireSphere(originNear, 0.1f);

        Vector3 fanOriginBase = transform.position + Vector3.up * obstacleSensorHeight +
transform.forward * obstacleSensorForwardOffset;
        int raysPerCol = Mathf.Max(1, obstacleFanRayCount / 3);
        float[] latOffs = { -obstacleFanLateralSpread, 0f, +obstacleFanLateralSpread };

        for (int col = 0; col < 3; col++)
        {
            Vector3 colOrig = fanOriginBase + transform.right * latOffs[col];

            for (int r = 0; r < raysPerCol; r++)
            {
```

```
            float ft = (raysPerCol > 1) ? (float)r / (raysPerCol - 1) : 0.5f;
            float angle = Mathf.Lerp(-obstacleFanHalfAngle, +obstacleFanHalfAngle, ft);
            Vector3 rayDir = Quaternion.AngleAxis(angle, Vector3.up) * transform.forward;

            Gizmos.color = new Color(1f, 0.3f, 0.3f, 0.6f);
            Gizmos.DrawRay(colOrig, rayDir * obstacleDetectDistance);
            Gizmos.DrawWireSphere(colOrig, obstacleSphereRadius);
        }
    }

    Gizmos.color = new Color(1f, 0.6f, 0f, 0.8f);
    for (int i = 0; i < sideSensorCount; i++)
    {
        float st = (sideSensorCount > 1) ? (float)i / (sideSensorCount - 1) : 0.5f;
        float fwd = Mathf.Lerp(-sideSensorLengthSpread, sideSensorLengthSpread, st);
        Vector3 o = transform.position + Vector3.up * sideSensorHeight + transform.forward * fwd;
        Gizmos.DrawRay(o, transform.right * sideSensorRange);
        Gizmos.DrawWireSphere(o, sideSensorRadius);
    }
  }
}
```

APPENDIX B

6.7 Camera Shift

```
using UnityEngine;

public class TimedMultiViewFollowCamera : MonoBehaviour
{
    [Header("Target")]
    public Transform target;

    [Header("Follow Settings")]
    public float positionSmoothTime = 0.18f;
    public float rotationSmoothSpeed = 8f;
    public float switchInterval = 6f;
    public float extraBlendFactor = 1.0f;
    public float fovLerpSpeed = 6f;

    [Header("Views")]
    public ViewPreset[] views;

    [System.Serializable]
    public class ViewPreset
    {
        public string name = "Follow";
        public Vector3 localOffset = new Vector3(0f, 3f, -8f);
        public Vector3 lookAtLocalPoint = new Vector3(0f, 1f, 0f);
        public float fieldOfView = 60f;
    }

    private int currentViewIndex = 0;
    private float viewTimer = 0f;
    private Vector3 positionVelocity;
    private Camera cam;

    void Awake()
    {
        cam = GetComponent<Camera>();
        if (cam == null) cam = Camera.main;
    }
```

```
    void Start()
    {
        if (views == null || views.Length == 0)
        {
            // Fall back to a few usable demo angles if nothing is set in the Inspector.
            SetDefaultViews();
        }

        if (currentViewIndex >= views.Length)
        {
            currentViewIndex = 0;
        }
    }

    void LateUpdate()
    {
        if (target == null || views == null || views.Length == 0) return;

        if (switchInterval > 0f)
        {
            viewTimer += Time.deltaTime;
            if (viewTimer >= switchInterval)
            {
                // Rotate through the presets automatically during the run.
                NextView();
            }
        }

        ViewPreset currentView = views[currentViewIndex];

        // Offsets are stored in target local space so each view stays aligned with the robot.
        Vector3 wantedPosition = target.TransformPoint(currentView.localOffset);
        float moveSmooth = Mathf.Max(0.01f, positionSmoothTime) / Mathf.Max(0.1f, extraBlendFactor);
        transform.position = Vector3.SmoothDamp(transform.position, wantedPosition, ref
positionVelocity, moveSmooth);

        Vector3 lookPoint = target.TransformPoint(currentView.lookAtLocalPoint);
        Vector3 lookDirection = lookPoint - transform.position;
```

```
        if (lookDirection.sqrMagnitude > 0.001f)
        {
            // Avoid trying to build a rotation from a near-zero direction vector.
            Quaternion wantedRotation = Quaternion.LookRotation(lookDirection.normalized, Vector3.up);
            float turnSpeed = Mathf.Max(0.1f, rotationSmoothSpeed) * Mathf.Max(0.1f, extraBlendFactor);
            transform.rotation = Quaternion.Slerp(transform.rotation, wantedRotation, turnSpeed *
Time.deltaTime);
        }

        if (cam != null && currentView.fieldOfView > 0f)
        {
            // Blend FOV so the switch between views feels less abrupt.
            cam.fieldOfView = Mathf.Lerp(cam.fieldOfView, currentView.fieldOfView, fovLerpSpeed *
Time.deltaTime);
        }
    }

    private void SetDefaultViews()
    {
        // Basic angles for testing: follow, higher follow, top, side and front.
        views = new ViewPreset[]
        {
            new ViewPreset { name = "Follow", localOffset = new Vector3(0f, 3f, -8f), lookAtLocalPoint =
new Vector3(0f, 1f, 0f), fieldOfView = 60f },
            new ViewPreset { name = "Higher", localOffset = new Vector3(0f, 5.5f, -10f), lookAtLocalPoint
= new Vector3(0f, 1f, 0f), fieldOfView = 55f },
            new ViewPreset { name = "Top", localOffset = new Vector3(0f, 14f, -2f), lookAtLocalPoint =
new Vector3(0f, 0.5f, 0f), fieldOfView = 60f },
            new ViewPreset { name = "Side", localOffset = new Vector3(7f, 3f, -2f), lookAtLocalPoint =
new Vector3(0f, 1f, 0f), fieldOfView = 60f },
            new ViewPreset { name = "Front", localOffset = new Vector3(0f, 2.5f, 7f), lookAtLocalPoint =
new Vector3(0f, 1f, 0f), fieldOfView = 65f }
        };
    }

    public void NextView()
    {
        if (views == null || views.Length == 0) return;

        currentViewIndex++;
```

```
        if (currentViewIndex >= views.Length)
        {
            // Wrap back to the first camera preset.
            currentViewIndex = 0;
        }

        viewTimer = 0f;
    }
}
```

6.8 Performance logger

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Reflection;
using System.Text;
using UnityEngine;
using UnityEngine.InputSystem;

public class PerformanceLogger : MonoBehaviour
{
    [Header("Run")]
    public string runLabel = "TestRun";
    public string trackName = "Unknown";
    [TextArea]
    public string notes = "";

    [Header("Logging")]
    public int logEveryNFrames = 1;
    public string outputFolder = "LogData";

    private MonoBehaviour controller;
    private Rigidbody rb;
    private Type controllerType;

    private StreamWriter frameWriter;
    private string frameFilePath;
    private string summaryFilePath;
    private bool summaryWritten = false;
```

```
    private float runStartTime = 0f;
    private int frameCount = 0;
    private int loggedFrameCount = 0;

    private float sumAbsLateralError = 0f;
    private float sumSquaredLateralError = 0f;
    private float maxAbsLateralError = 0f;
    private int errorSampleCount = 0;

    private float sumSpeed = 0f;
    private float maxSpeed = 0f;

    private int offTrackCount = 0;
    private bool wasOffTrack = false;

    private int lapCount = 0;
    private float lastLapTime = 0f;
    private List<float> lapTimes = new List<float>();

    private float totalTimeInCurve = 0f;
    private float totalTimeRecovering = 0f;

    void Start()
    {
        controller = GetComponent("RobotController") as MonoBehaviour;
        rb = GetComponent<Rigidbody>();

        if (controller == null)
        {
            Debug.LogError("[PerformanceLogger] RobotController not found!");
            enabled = false;
            return;
        }

        if (rb == null)
        {
            Debug.LogError("[PerformanceLogger] Rigidbody not found!");
            enabled = false;
            return;
        }
```

```
        controllerType = controller.GetType();
        logEveryNFrames = Mathf.Max(1, logEveryNFrames);

        SetupOutputFiles();
        ResetRunStats();

        Debug.Log("[PerformanceLogger] Started logging to: " + frameFilePath);
    }

    void FixedUpdate()
    {
        if (controller == null || rb == null || frameWriter == null) return;

        frameCount++;

        // Use reflection so the logger still works even if the controller version changes.
        float lateralError = ReadFirstFloat("smoothedError", "currentLateralError", "currentError");
        float headingError = ReadFirstFloat("smoothedHeadingError", "currentHeadingError");
        float turnRate = ReadFirstFloat("appliedTurnRate", "currentTurnRate", "rateLimitedTurnRate");
        float targetSpeed = ReadFirstFloat("smoothedTargetVelocity", "targetVelocity");

        bool isInCurve = ReadBool("isInCurve");
        bool isRecovering = ReadBool("isRecovering");
        bool leftDetected = ReadBool("leftLineDetected");
        bool rightDetected = ReadBool("rightLineDetected");

        float pTerm = ReadFirstFloat("debugPTerm");
        float iTerm = ReadFirstFloat("debugITerm");
        float dTerm = ReadFirstFloat("debugDTerm");
        float hTerm = ReadFirstFloat("debugHTerm");
        float ffTerm = ReadFirstFloat("debugFFTerm");

        string avoidState = ReadFieldAsString("avoidState");
        if (string.IsNullOrEmpty(avoidState))
            avoidState = "None";

        float speed = rb.linearVelocity.magnitude;
        float forwardSpeed = Vector3.Dot(rb.linearVelocity, transform.forward);
        float absError = Mathf.Abs(lateralError);
```

```
sumAbsLateralError += absError;
sumSquaredLateralError += lateralError * lateralError;
if (absError > maxAbsLateralError) maxAbsLateralError = absError;
errorSampleCount++;

sumSpeed += speed;
if (speed > maxSpeed) maxSpeed = speed;

bool currentlyOffTrack = !leftDetected && !rightDetected;
if (currentlyOffTrack && !wasOffTrack)
    offTrackCount++;
wasOffTrack = currentlyOffTrack;

if (isInCurve) totalTimeInCurve += Time.fixedDeltaTime;
if (isRecovering) totalTimeRecovering += Time.fixedDeltaTime;

if (frameCount % logEveryNFrames != 0) return;

loggedFrameCount++;
float elapsed = Time.time - runStartTime;

// Older controller versions may not expose turn rate directly, so use rigidbody yaw as a fallback.
if (Mathf.Abs(turnRate) < 0.0001f && Mathf.Abs(rb.angularVelocity.y) > 0.001f)
    turnRate = rb.angularVelocity.y;

frameWriter.WriteLine(
    $"{elapsed:F4}," +
    $"{transform.position.x:F4},{transform.position.z:F4}," +
    $"{speed:F4}," +
    $"{forwardSpeed:F4}," +
    $"{lateralError:F6}," +
    $"{headingError:F6}," +
    $"{turnRate:F6}," +
    $"{pTerm:F6},{iTerm:F6},{dTerm:F6},{hTerm:F6},{ffTerm:F6}," +
    $"{(isInCurve ? 1 : 0)},{(isRecovering ? 1 : 0)}," +
    $"{ToCsvCell(avoidState)}," +
    $"{(leftDetected ? 1 : 0)},{(rightDetected ? 1 : 0)}," +
    $"{targetSpeed:F4}," +
    $"{lapCount}"
```

```
        );
    }

    void Update()
    {
        if (Keyboard.current == null) return;

        // Lap marks are manual so runs can be tested on different track layouts.
        if (Keyboard.current.lKey.wasPressedThisFrame)
        {
            lapCount++;

            float now = Time.time;
            float lapTime = now - lastLapTime;
            lapTimes.Add(lapTime);
            lastLapTime = now;

            Debug.Log($"[PerformanceLogger] Lap {lapCount} completed. Time: {lapTime:F2}s");
        }
    }

    void OnDestroy()
    {
        WriteSummary();
        CloseWriter();
    }

    void OnApplicationQuit()
    {
        WriteSummary();
        CloseWriter();
    }

    private void SetupOutputFiles()
    {
        string basePath = Path.Combine(Application.dataPath, "..", outputFolder);
        Directory.CreateDirectory(basePath);

        string timestamp = DateTime.Now.ToString("yyyyMMdd_HHmmss");
        string safeLabel = MakeSafeFileLabel(runLabel);
```

```
        frameFilePath = Path.Combine(basePath, safeLabel + "_" + timestamp + ".csv");
        summaryFilePath = Path.Combine(basePath, "RunSummary.csv");

        frameWriter = new StreamWriter(frameFilePath, false, Encoding.UTF8);
        frameWriter.WriteLine(
            "Time," +
            "PosX,PosZ," +
            "Speed," +
            "ForwardSpeed," +
            "LateralError," +
            "HeadingError," +
            "TurnRate," +
            "PTerm,ITerm,DTerm,HTerm,FFTerm," +
            "IsInCurve,IsRecovering," +
            "AvoidState," +
            "LeftDetected,RightDetected," +
            "TargetSpeed," +
            "LapCount"
        );
    }

    private void ResetRunStats()
    {
        runStartTime = Time.time;
        frameCount = 0;
        loggedFrameCount = 0;

        sumAbsLateralError = 0f;
        sumSquaredLateralError = 0f;
        maxAbsLateralError = 0f;
        errorSampleCount = 0;

        sumSpeed = 0f;
        maxSpeed = 0f;

        offTrackCount = 0;
        wasOffTrack = false;

        lapCount = 0;
```

```
        lapTimes.Clear();
        lastLapTime = Time.time;

        totalTimeInCurve = 0f;
        totalTimeRecovering = 0f;
        summaryWritten = false;
    }

    private void CloseWriter()
    {
        if (frameWriter != null)
        {
            frameWriter.Flush();
            frameWriter.Close();
            frameWriter = null;
        }

        Debug.Log($"[PerformanceLogger] Stopped. {loggedFrameCount} frames written.");
    }

    private void WriteSummary()
    {
        if (summaryWritten || errorSampleCount == 0) return;
        summaryWritten = true;

        float totalTime = Time.time - runStartTime;
        float meanAbsError = sumAbsLateralError / errorSampleCount;
        float rmsError = Mathf.Sqrt(sumSquaredLateralError / errorSampleCount);
        float meanSpeed = sumSpeed / errorSampleCount;

        float kp = ReadFirstFloat(true, "Kp");
        float ki = ReadFirstFloat(true, "Ki");
        float kd = ReadFirstFloat(true, "Kd");
        float kh = ReadFirstFloat(true, "Kh");
        float kff = ReadFirstFloat(true, "Kff");

        string lapTimesStr = lapTimes.Count > 0
            ? string.Join(";", lapTimes.ConvertAll(t => t.ToString("F2")))
            : "N/A";
```

```
        bool writeHeader = !File.Exists(summaryFilePath);

        // The summary file is append-only so multiple runs can be compared later.
        using (StreamWriter sw = new StreamWriter(summaryFilePath, true, Encoding.UTF8))
        {
            if (writeHeader)
            {
                sw.WriteLine(
                    "RunLabel,TrackName,DateTime," +
                    "Kp,Ki,Kd,Kh,Kff," +
                    "TotalTime_s,Laps,LapTimes_s," +
                    "MeanAbsError,RMS_Error,MaxAbsError," +
                    "MeanSpeed,MaxSpeed," +
                    "OffTrackEvents," +
                    "TimeInCurves_s,TimeRecovering_s," +
                    "TotalFrames,Notes"
                );
            }

            sw.WriteLine(
                $"{ToCsvCell(runLabel)},{ToCsvCell(trackName)},{DateTime.Now:yyyy-MM-dd HH:mm:ss},"
+
                $"{kp:F3},{ki:F4},{kd:F3},{kh:F3},{kff:F3}," +
                $"{totalTime:F2},{lapCount},{ToCsvCell(lapTimesStr)}," +
                $"{meanAbsError:F6},{rmsError:F6},{maxAbsLateralError:F6}," +
                $"{meanSpeed:F4},{maxSpeed:F4}," +
                $"{offTrackCount}," +
                $"{totalTimeInCurve:F2},{totalTimeRecovering:F2}," +
                $"{loggedFrameCount},{ToCsvCell(notes)}"
            );
        }

        Debug.Log(
            $"[PerformanceLogger] Summary saved. MAE:{meanAbsError:F4} RMS:{rmsError:F4} " +
            $"Max:{maxAbsLateralError:F4} OffTrack:{offTrackCount}"
        );
    }

    private float ReadFirstFloat(params string[] fieldNames)
    {
```

```
    return ReadFirstFloat(false, fieldNames);
}

private float ReadFirstFloat(bool publicOnly, params string[] fieldNames)
{
    for (int i = 0; i < fieldNames.Length; i++)
    {
        if (TryReadFloat(fieldNames[i], out float value, publicOnly))
            return value;
    }

    return 0f;
}

private bool TryReadFloat(string fieldName, out float value, bool publicOnly = false)
{
    value = 0f;

    BindingFlags flags = publicOnly
        ? BindingFlags.Public | BindingFlags.Instance
        : BindingFlags.Public | BindingFlags.NonPublic | BindingFlags.Instance;

    FieldInfo field = controllerType.GetField(fieldName, flags);
    if (field == null || field.FieldType != typeof(float))
        return false;

    value = (float)field.GetValue(controller);
    return true;
}

private bool ReadBool(string fieldName)
{
    BindingFlags flags = BindingFlags.Public | BindingFlags.NonPublic | BindingFlags.Instance;
    FieldInfo field = controllerType.GetField(fieldName, flags);

    if (field != null && field.FieldType == typeof(bool))
        return (bool)field.GetValue(controller);

    return false;
}
```

```
    private string ReadFieldAsString(string fieldName)
    {
        BindingFlags flags = BindingFlags.Public | BindingFlags.NonPublic | BindingFlags.Instance;
        FieldInfo field = controllerType.GetField(fieldName, flags);

        if (field == null) return "";

        object value = field.GetValue(controller);
        return value != null ? value.ToString() : "";
    }

    private string MakeSafeFileLabel(string value)
    {
        if (string.IsNullOrWhiteSpace(value))
            return "Run";

        string safe = value.Trim().Replace(" ", "_").Replace("/", "-").Replace("\\", "-");
        char[] invalidChars = Path.GetInvalidFileNameChars();

        for (int i = 0; i < invalidChars.Length; i++)
        {
            safe = safe.Replace(invalidChars[i].ToString(), "");
        }

        return string.IsNullOrWhiteSpace(safe) ? "Run" : safe;
    }

    private string ToCsvCell(string value)
    {
        if (string.IsNullOrEmpty(value))
            return "\"\"";

        string escaped = value.Replace("\"", "\"\"");
        return "\"" + escaped + "\"";
    }
}
```

6.9 Simple Object Movement

```
using UnityEngine;
```

```
public class BackAndForth : MonoBehaviour
{
    [Header("Movement")]
    public float moveSpeed = 2f;
    public float moveTime = 2f;

    [Header("Turning")]
    public float spinTime = 0.5f;

    private float timer = 0f;
    private bool goingForward = true;
    private bool isSpinning = false;

    private Quaternion spinStart;
    private Quaternion spinEnd;

    void Update()
    {
        if (isSpinning)
        {
            timer += Time.deltaTime;
            float t = Mathf.Clamp01(timer / Mathf.Max(0.01f, spinTime));
            transform.localRotation = Quaternion.Slerp(spinStart, spinEnd, t);

            if (t >= 1f)
            {
                isSpinning = false;
                goingForward = !goingForward;
                timer = 0f;
            }

            return;
        }

        // The object moves in local forward space, so after the 180 turn it drives back along the same
path.
        transform.Translate(Vector3.forward * moveSpeed * Time.deltaTime, Space.Self);

        timer += Time.deltaTime;
```

```
        if (timer >= moveTime)
        {
            // Rotate in place before starting the return part of the movement.
            isSpinning = true;
            timer = 0f;
            spinStart = transform.localRotation;
            spinEnd = spinStart * Quaternion.Euler(0f, 180f, 0f);
        }
    }
}
```

6.10 Wheel Movement

```
using UnityEngine;

// Simple script to rotate wheels visually based on the robot's movement
// Attach this to Car8 object

public class SimpleWheelRotation : MonoBehaviour
{
    [Header("Wheel References")]
    [Tooltip("Assign all wheel objects that should rotate")]
    public Transform[] wheels;

    [Header("Rotation Settings")]
    [Tooltip("Speed multiplier for wheel rotation")]
    public float rotationSpeed = 500f;

    [Tooltip("Reference to the Robot (parent) to get velocity")]
    public Transform robotTransform;

    private Rigidbody robotRigidbody;

    void Start()
    {
        // Auto-find the robot if not assigned
        if (robotTransform == null)
        {
            robotTransform = transform.parent;
        }
```

```
        if (robotTransform != null)
        {
            robotRigidbody = robotTransform.GetComponent<Rigidbody>();
        }

        if (wheels.Length == 0)
        {
            Debug.LogWarning("No wheels assigned! Please assign wheel transforms in the Inspector.");
        }

        Debug.Log("SimpleWheelRotation initialized!");
    }

    void Update()
    {
        if (wheels.Length == 0) return;

        // Get the robot's velocity
        float speed = 0f;
        if (robotRigidbody != null)
        {
            speed = robotRigidbody.linearVelocity.magnitude;
        }

        // Calculate rotation amount based on speed
        float rotationAmount = speed * rotationSpeed * Time.deltaTime;

        // Rotate all wheels around their local X axis
        foreach (Transform wheel in wheels)
        {
            if (wheel != null)
            {
                wheel.Rotate(rotationAmount, 0, 0, Space.Self);
            }
        }
    }
}
```